

STAR RX72N - rx_i2c_comm Library
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 libs/rx_i2c_comm/inc Directory Reference	5
3.2 libs Directory Reference	5
3.3 libs/rx_i2c_comm Directory Reference	6
3.4 libs/rx_i2c_comm/src Directory Reference	6
4 File Documentation	7
4.1 libs/rx_i2c_comm/inc/rx_i2c_comm.h File Reference	7
4.1.1 Detailed Description	8
4.1.2 Data Structure Documentation	11
4.1.2.1 struct rx_i2c_comm_handle_t	11
4.1.2.2 struct rx_i2c_comm_config_t	14
4.1.3 Enumeration Type Documentation	15
4.1.3.1 rx_i2c_comm_constants_t	15
4.1.3.2 rx_i2c_comm_defaults_t	16
4.1.4 Function Documentation	17
4.1.4.1 rx_i2c_comm_deinit()	17
4.1.4.2 rx_i2c_comm_init()	18
4.1.4.3 rx_i2c_comm_receive()	20
4.1.4.4 rx_i2c_comm_send()	22
4.2 rx_i2c_comm.h	24
4.3 libs/rx_i2c_comm/src/rx_i2c_comm.c File Reference	30
4.3.1 Detailed Description	32
4.3.2 Enumeration Type Documentation	34
4.3.2.1 i2c_crc32_seed_t	34
4.3.2.2 i2c_dispatch_result_t	35
4.3.2.3 i2c_frame_header_offset_t	35
4.3.2.4 rx_i2c_comm_header_size_t	35
4.3.2.5 rx_i2c_comm_sequence_constants_t	36
4.3.2.6 rx_i2c_comm_sync_constants_t	36
4.3.2.7 rx_i2c_comm_sync_result_t	36
4.3.2.8 rx_receive_result_t	37
4.3.3 Function Documentation	37
4.3.3.1 internal_align_to_sync()	37
4.3.3.2 internal_build_frame()	38
4.3.3.3 internal_compact_rx_buffer()	39
4.3.3.4 internal_decode_frame()	40
4.3.3.5 internal_decode_header()	41

4.3.3.6	<code>internal_dispatch_frame()</code>	42
4.3.3.7	<code>internal_find_sync()</code>	43
4.3.3.8	<code>internal_handle_no_sync()</code>	44
4.3.3.9	<code>internal_handle_ping()</code>	45
4.3.3.10	<code>internal_handle_reset()</code>	46
4.3.3.11	<code>internal_parse_header()</code>	47
4.3.3.12	<code>internal_read_i2c_data()</code>	48
4.3.3.13	<code>internal_receive_iteration()</code>	49
4.3.3.14	<code>internal_validate_wire_len()</code>	51
4.3.3.15	<code>internal_verify_crc()</code>	52
4.3.3.16	<code>rx_i2c_comm_deinit()</code>	53
4.3.3.17	<code>rx_i2c_comm_init()</code>	54
4.3.3.18	<code>rx_i2c_comm_receive()</code>	56
4.3.3.19	<code>rx_i2c_comm_send()</code>	58
4.3.4	Variable Documentation	60
4.3.4.1	<code>s_tag</code>	60
4.3.4.2	<code>s_zero_handle</code>	60
4.4	<code>rx_i2c_comm.c</code>	61

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_i2c_comm.h	7
libs	5
rx_i2c_comm	6
inc	5
rx_i2c_comm.h	7
src	6
rx_i2c_comm.c	30
rx_i2c_comm	6
inc	5
rx_i2c_comm.h	7
src	6
rx_i2c_comm.c	30
src	6
rx_i2c_comm.c	30

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

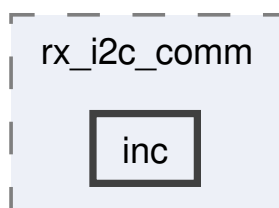
libs/rx_i2c_comm/inc/ rx_i2c_comm.h	
High-Level I2C Peripheral Communication Layer for RX72N	7
libs/rx_i2c_comm/src/ rx_i2c_comm.c	
High-Level I2C Peripheral Communication Layer Implementation	30

Chapter 3

Directory Documentation

3.1 libs/rx_i2c_comm/inc Directory Reference

Directory dependency graph for inc:

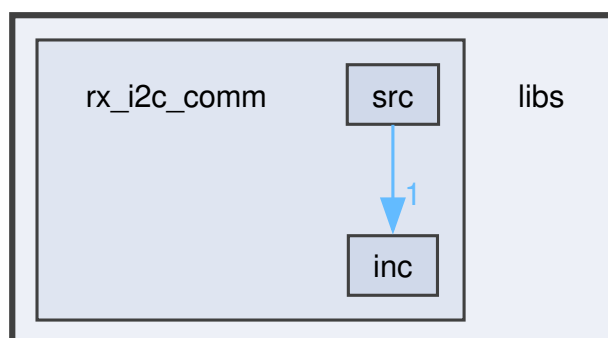


Files

- file [rx_i2c_comm.h](#)
High-Level I2C Peripheral Communication Layer for RX72N.

3.2 libs Directory Reference

Directory dependency graph for libs:

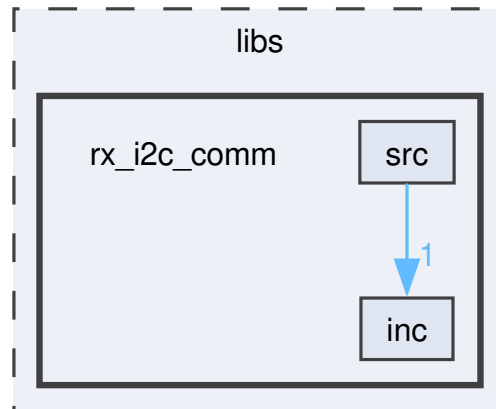


Directories

- directory [rx_i2c_comm](#)

3.3 libs/rx_i2c_comm Directory Reference

Directory dependency graph for rx_i2c_comm:

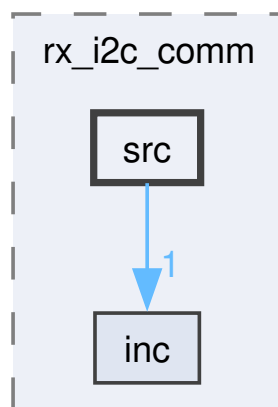


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_i2c_comm/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_i2c_comm.c](#)
High-Level I2C Peripheral Communication Layer Implementation.

Chapter 4

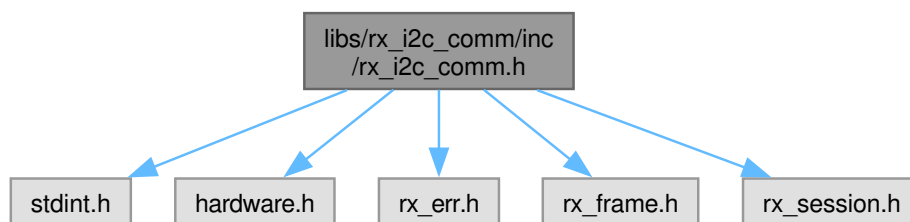
File Documentation

4.1 libs/rx_i2c_comm/inc/rx_i2c_comm.h File Reference

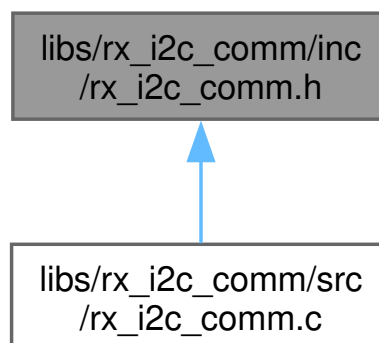
High-Level I2C Peripheral Communication Layer for RX72N.

```
#include <stdint.h>
#include "hardware.h"
#include "rx_err.h"
#include "rx_frame.h"
#include "rx_session.h"
```

Include dependency graph for rx_i2c_comm.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct `rx_i2c_comm_handle_t`
I2C communication handle containing all operational state.
- struct `rx_i2c_comm_config_t`
I2C communication configuration options.

Enumerations

- enum `rx_i2c_comm_constants_t` : `uint16_t` { `k_i2c_comm_rx_buffer_size` = 2048 , `k_i2c_comm_tx_buffer_size` = 2048 , `k_i2c_comm_max_receive_iterations` = 1000 }
I2C communication configuration constants.
- enum `rx_i2c_comm_defaults_t` : `uint8_t` { `k_i2c_comm_default_channel` = 0 , `k_i2c_comm_default_addr` = 0x42 }
Default I2C channel and address for RPi5 communication.

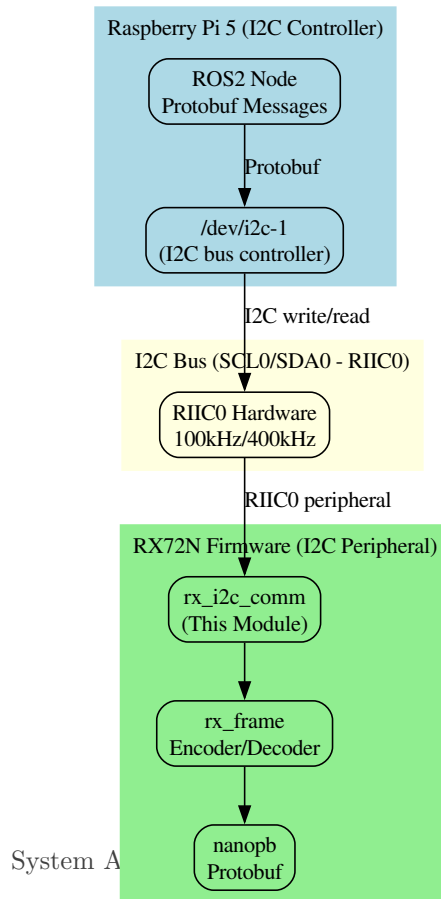
Functions

- `rx_err_t rx_i2c_comm_init (rx_i2c_comm_handle_t *handle, const rx_i2c_comm_config_t *config)`
Initialize I2C communication handle in peripheral mode.
- `rx_err_t rx_i2c_comm_deinit (rx_i2c_comm_handle_t *handle)`
Deinitialize I2C communication handle.
- `rx_err_t rx_i2c_comm_send (rx_i2c_comm_handle_t *handle, rx_frame_type_t type, uint8_t flags, const uint8_t *payload, uint32_t payload_len)`
Send a frame over I2C (RIIC peripheral -> controller).
- `rx_err_t rx_i2c_comm_receive (rx_i2c_comm_handle_t *handle, rx_frame_t *frame, uint32_t timeout_ms)`
Receive and decode a frame from I2C controller (RPi5 write).

4.1.1 Detailed Description

High-Level I2C Peripheral Communication Layer for RX72N.

Integrates the Frame Layer and RIIC0 (I2C) peripheral driver for reliable communication with the RPi5 controller acting as I2C bus controller. The RX72N acts as an I2C peripheral (device), responding to a fixed 7-bit address set during initialization. This provides an alternative communication channel when SPI or USB CDC are unavailable.



Peripheral Mode Operation

The RX72N acts as an I2C peripheral device at a configurable 7-bit address. The RPi5 initiates all transactions as I2C controller. The RX72N uses polling (ICSR2.RDRF for receive, ICSR2.TDRE for transmit) to handle incoming writes and prepare outgoing read responses.

Features

- Frame encoding/decoding with CRC-32 error detection
- Automatic sequence number tracking for reliable delivery
- Non-blocking receive operations
- Same wire protocol as USB/SPI/UART (interchangeable transport)
- Configurable I2C channel and device address

Frame Protocol

Uses the same frame format as USB, SPI, and UART communication:

SYNC	TYPE	FLAG	SEQ	LENGTH	PAYLOAD	CRC32
0xAA	1B	1B	2B	2B	N bytes	4B

I2C Channel Assignment

Channel	Peripheral	Addr	Purpose
RIIC0	k_i2c_comm_default_channel	0x42 (default)	Binary protocol (nanopb)

Performance Characteristics

Metric	Value	Notes
I2C speed	400kHz max	Fast mode supported
Frame latency	~1-5 ms	Depends on I2C clock speed
Max frame size	2048 bytes	Buffer size limit

Memory Usage

Component	Size	Description
rx_i2c_comm_handle_t	~4.2 KB	Per-instance handle
RX buffer	2048 B	Staging buffer for receives
TX buffer	2048 B	Staging buffer for sends

Thread Safety

The handle is NOT thread-safe. External synchronization (ThreadX mutex) is required for concurrent access from multiple threads.

Module Dependencies

- rx_frame.h: Frame encoding/decoding layer
- hardware.h: I2C HAL (riic_init_peripheral, riic_peripheral_read, riic_peripheral_write)
- rx_err.h: Error code definitions
- rx_session.h: Cross-transport sequence continuity

NASA Power of 10 Compliance

- Rule 1: [PASS] No goto, setjmp, or recursion
- Rule 2: [PASS] Loop bounds via k_i2c_comm_max_receive_iterations
- Rule 3: [PASS] No dynamic memory (static buffers in handle)
- Rule 4: [PASS] Functions are concise
- Rule 5: [PASS] Minimum 2 pre/post conditions per public function
- Rule 6: [PASS] Variables declared at smallest scope
- Rule 7: [PASS] All return values checked
- Rule 8: [PASS] Constants use C23 typed enums
- Rule 10: [PASS] Compiles with -Wall -Wextra -Werror

SOLID Principles

- S (SRP): I2C communication only, frame details in `rx_frame`
- O (OCP): Configurable channel/address via [rx_i2c_comm_config_t](#)
- L (LSP): Same protocol as `rx_usb_comm` / `rx_uart_comm` (interchangeable)
- I (ISP): Clean send/receive interface
- D (DIP): Depends on hardware.h HAL abstraction

See also

`rx_frame.h` Frame layer implementation

`hardware.h` I2C HAL

`rx_usb_comm.h` USB CDC communication (same protocol)

`rx_uart_comm.h` UART communication (same protocol)

`docs/sections/01_nanopb_protocol.tex` Protocol specification

Author

Locked, Inc.

Date

2026-03-11

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Definition in file [rx_i2c_comm.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct `rx_i2c_comm_handle_t`

I2C communication handle containing all operational state.

Contains all state for I2C peripheral communication including frame encoder/decoder, sequence tracking, and staging buffers. One handle corresponds to one logical I2C communication channel (RIIC0).

Memory Layout

Offset	Size	Field	Description
0	~50 B	encoder	Frame encoder state machine
50	~50 B	decoder	Frame decoder state machine
100	2048 B	rx_buffer	RX staging buffer
2148	2048 B	tx_buffer	TX staging buffer
4196	4 B	rx_buffer_len	Valid bytes in RX buffer
4200	4 B	rx_buffer_pos	Current read position
4204	4-8 B	session	Shared session state pointer
4212	1 B	initialized	Init flag
4213	1 B	channel_value	RIIC channel number
4214	1 B	device_addr	7-bit I2C device address
Total	**~4.2 KB**		

Invariant

```

initialized == 0 or initialized == 1
rx_buffer_pos <= rx_buffer_len
rx_buffer_len <= k_i2c_comm_rx_buffer_size

```

Warning

Do not access fields directly; use API functions

Attention

Handle must be initialized via [rx_i2c_comm_init\(\)](#) before use

See also

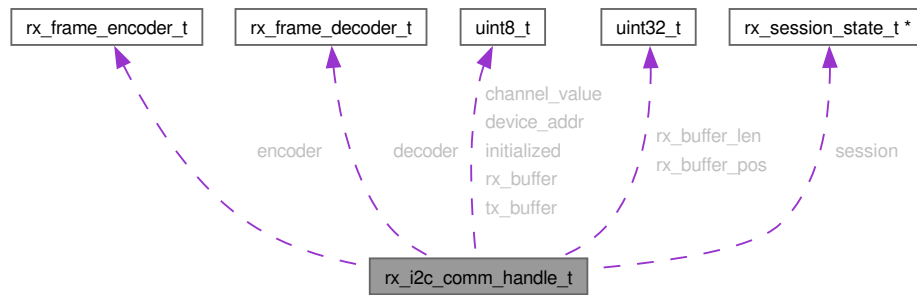
[rx_i2c_comm_init\(\)](#) Initialize handle
[rx_i2c_comm_deinit\(\)](#) Cleanup handle
[rx_i2c_comm_config_t](#) Configuration options

Since

Version 1.0.0

Definition at line 257 of file [rx_i2c_comm.h](#).

Collaboration diagram for [rx_i2c_comm_handle_t](#):



Data Fields

uint8_t	channel_value	RIIC channel number. Stores the riic_channel_t.value configured during init.
rx_frame_decoder_t	decoder	Frame decoder state machine. Decodes incoming bytes, validates CRC, extracts payload.
uint8_t	device_addr	7-bit I2C device address for this peripheral Stores the i2c_device_addr_t.value configured during init.
rx_frame_encoder_t	encoder	Frame encoder state machine. Encodes payloads into framed format with sync, header, and CRC.
uint8_t	initialized	Initialization status flag. Values: 0 = not initialized, 1 = initialized
uint8_t	rx_buffer[k_i2c_comm_rx_buffer_size]	RX staging buffer. Accumulates incoming I2C data before frame decoding. Invariant Valid data in [0, rx_buffer_len)
uint32_t	rx_buffer_len	Number of valid bytes in RX buffer. Invariant rx_buffer_len <= k_i2c_comm_rx_buffer_size

uint32_t	rx_buffer_pos	Current read position in RX buffer. Invariant rx_buffer_pos <= rx_buffer_len
rx_session_state_t *	session	Shared session state for cross-transport sequence continuity. Points to session state owned by rx_comm_manager. See also rx_session_state_t
uint8_t	tx_buffer[k_i2c_comm_tx_buffer_size]	TX staging buffer. Holds assembled frame for I2C transmission.

4.1.2.2 struct rx_i2c_comm_config_t

I2C communication configuration options.

Configuration passed to [rx_i2c_comm_init\(\)](#). The session pointer is required for sequence tracking.

Default Values

Field	Default	Description
session	(required)	Shared session state pointer
channel	k_i2c_comm_default_channel	RIIC0 is the default I2C channel
device_addr	k_i2c_comm_default_addr	0x42 default RPi5 peripheral address

See also

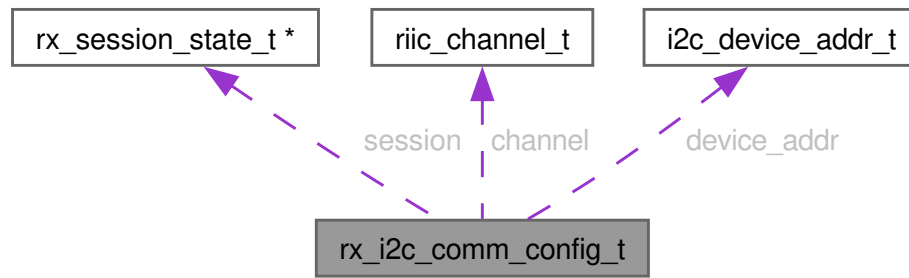
[rx_i2c_comm_init\(\)](#) Uses this configuration

Since

Version 1.0.0

Definition at line 340 of file [rx_i2c_comm.h](#).

Collaboration diagram for rx_i2c_comm_config_t:



Data Fields

riic_channel_t	channel	RIIC channel to use. Selects which RIIC peripheral to use for I2C communication. Default: {.value = k_i2c_comm_default_channel} (RIIC0)
i2c_device_addr_t	device_addr	7-bit I2C device address for this RX72N peripheral RPi5 will address this device on the I2C bus. Default: {.value = k_i2c_comm_default_addr} (0x42)
rx_session_state_t *	session	Shared session state for cross-transport sequence tracking. Must be initialized via rx_session_init() before passing here. Required - must not be NULL.

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_i2c_comm_constants_t

```
enum rx_i2c_comm_constants_t : uint16_t
```

I2C communication configuration constants.

Compile-time constants for I2C communication buffer sizes and loop bounds. These values are tuned for the RX72N RIIC0 I2C implementation with typical control message sizes.

Buffer Sizing Rationale

- 2048 bytes accommodates largest expected protobuf message + frame overhead
- Provides headroom for frame header (8 bytes) and CRC (4 bytes)

Invariant

Buffer sizes must be $\geq$ maximum frame size

k_i2c_comm_max_receive_iterations bounds all receive loops

Since

Version 1.0.0

Enumerator

<code>k_i2c_comm_rx_buffer_size</code>	RX staging buffer size in bytes. Buffer for accumulating incoming I2C data before frame decoding. Value: 2048 bytes
<code>k_i2c_comm_tx_buffer_size</code>	TX staging buffer size in bytes. Buffer for assembled frames before I2C transmission. Value: 2048 bytes
<code>k_i2c_comm_max_receive_iterations</code>	Maximum iterations for receive loop (NASA Rule 2). Provides static upper bound for receive polling loops to ensure bounded execution time even under pathological conditions. Value: 1000 iterations

Definition at line 168 of file [rx_i2c_comm.h](#).

```

00168         : uint16_t {
00169     /**
00170      * @brief RX staging buffer size in bytes
00171      * @details
00172      * Buffer for accumulating incoming I2C data before frame decoding.
00173      * @par Value: 2048 bytes
00174      */
00175     k_i2c_comm_rx_buffer_size = 2048,
00176
00177     /**
00178      * @brief TX staging buffer size in bytes
00179      * @details
00180      * Buffer for assembled frames before I2C transmission.
00181      * @par Value: 2048 bytes
00182      */
00183     k_i2c_comm_tx_buffer_size = 2048,
00184
00185     /**
00186      * @brief Maximum iterations for receive loop (NASA Rule 2)
00187      * @details
00188      * Provides static upper bound for receive polling loops to ensure
00189      * bounded execution time even under pathological conditions.
00190      * @par Value: 1000 iterations
00191      */
00192     k_i2c_comm_max_receive_iterations = 1000,
00193 } rx_i2c_comm_constants_t;

```

4.1.3.2 `rx_i2c_comm_defaults_t`

```
enum rx\_i2c\_comm\_defaults\_t : uint8_t
```

Default I2C channel and address for RPi5 communication.

Default hardware channel (RIIC0) and 7-bit I2C device address (0x42) used when the caller does not override via [rx_i2c_comm_config_t](#).

Address Selection

0x42 ('B' in ASCII) is chosen to avoid conflicts with common I2C sensor addresses (0x20-0x27 GPIO expanders, 0x3C/0x3D OLED, 0x48-0x4F ADC/temp, 0x68/0x69 IMU, 0x76/0x77 barometer).

Since

Version 1.0.0

Enumerator

k_i2c_comm_default_channel	RIIC0 channel
k_i2c_comm_default_addr	Default 7-bit device address

Definition at line 210 of file [rx_i2c_comm.h](#).

```
00210      : uint8_t {
00211    k_i2c_comm_default_channel = 0,    /**< RIIC0 channel */
00212    k_i2c_comm_default_addr   = 0x42, /**< Default 7-bit device address */
00213 } rx_i2c_comm_defaults_t;
```

4.1.4 Function Documentation

4.1.4.1 rx_i2c_comm_deinit()

```
rx_err_t rx_i2c_comm_deinit (
    rx\_i2c\_comm\_handle\_t * handle) [nodiscard]
```

Deinitialize I2C communication handle.

Clears the initialized flag to prevent further use of the handle. Does not deinitialize the underlying RIIC hardware.

Parameters

in,out	handle	Pointer to initialized handle
--------	--------	-------------------------------

Returns

k_rx_ok on success, k_rx_err_invalid_arg if handle is nullptr

Precondition

handle must not be nullptr

handle must have been initialized via [rx_i2c_comm_init\(\)](#)

Postcondition

handle->initialized == 0

Further calls to rx_i2c_comm_send/receive will return k_rx_err_invalid_state

Since

Version 1.0.0

Definition at line 685 of file [rx_i2c_comm.c](#).

```
00686 {
00687     if (handle == nullptr) {
00688         return k_rx_err_invalid_arg;
00689     }
00690     RX_ASSERT_PRE(handle->initialized, "Attempt to deinitialize uninitialized I2C comm handle");
00691     (void)rx_frame_encoder_deinit(&handle->encoder);
00692     (void)rx_frame_decoder_deinit(&handle->decoder);
00693     /* Hardware (RIIC peripheral) cleanup is the caller's responsibility.
00694     * The hardware.h HAL does not expose a riic_deinit() function; the RIIC
00695     * module stop bit (MSTPCRB) must be set by the caller if power-gating is
00696     * needed after deinitializing this comm layer. */
00700     handle->initialized = 0U;
00701     rx_log_debug(s_tag, "I2C comm deinitialized");
00702     return k_rx_ok;
00703 }
00704
00705 }
```

References [rx_i2c_comm_handle_t::decoder](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initialized](#), and [s_tag](#).

4.1.4.2 rx_i2c_comm_init()

```
rx_err_t rx_i2c_comm_init (
    rx_i2c_comm_handle_t * handle,
    const rx_i2c_comm_config_t * config) [nodiscard]
```

Initialize I2C communication handle in peripheral mode.

Sets up frame encoder/decoder, clears internal buffers, configures RIIC peripheral at the specified I2C device address, and prepares the handle for communication.

Algorithm

1. Validate handle pointer (NULL check)
2. Clear handle memory
3. Initialize frame encoder and decoder
4. Copy configuration (channel, device_addr, session)
5. Call `riic_init_peripheral()` to configure hardware
6. Set initialized flag

Parameters

out	handle	Pointer to handle to initialize (must not be nullptr)
in	config	Configuration options (must not be NULL) • session: Required shared session state pointer • channel: RIIC channel (default RIIC0) • device_addr: 7-bit I2C device address

Returns

rx_err_t Error code

Return values

k_rx_ok	Handle initialized successfully
k_rx_err_invalid_arg	handle or config is nullptr, or session is nullptr
Other	errors from riic_init_peripheral() or frame encoder/decoder init

Precondition

handle points to allocated [rx_i2c_comm_handle_t](#)
 config->session points to initialized rx_session_state_t
 config->device_addr.value must be in range [0x08, 0x77]

Postcondition

handle->initialized == 1
 RIIC peripheral will respond to config->device_addr on I2C bus
 handle->encoder and handle->decoder are in their initial reset state

Note

Not thread-safe; call from single initialization context
 Only initializes software handle; RIIC hardware configured via riic_init_peripheral()

See also

[rx_i2c_comm_deinit\(\)](#) Cleanup resources
[riic_channel_t](#) I2C channel wrapper
[i2c_device_addr_t](#) I2C device address wrapper

Since

Version 1.0.0

Definition at line 645 of file [rx_i2c_comm.c](#).

```

00646 {
00647     if (handle == nullptr) {
00648         return k_rx_err_invalid_arg;
00649     }
00650
00651     *handle = s_zero_handle;
00652
00653     if (config == nullptr || config->session == nullptr) {
00654         return k_rx_err_invalid_arg;
00655     }
00656     handle->session = config->session;
00657     handle->channel_value = config->channel.value;
00658     handle->device_addr = config->device_addr.value;
00659
00660     /* encoder/decoder init only fail for nullptr, which is impossible since handle is non-null */
00661     (void)rx_frame_encoder_init(&handle->encoder);
00662     (void)rx_frame_decoder_init(&handle->decoder);
00663

```

```

00664  /* Initialize RIIC in peripheral mode */
00665  rx_err_t err = riic_init_peripheral(config->channel, config->device_addr);
00666  if (err != k_rx_ok) {
00667      rx_log_error(s_tag, "Failed to init RIIC peripheral mode");
00668      (void)rx_frame_encoder_deinit(&handle->encoder);
00669      (void)rx_frame_decoder_deinit(&handle->decoder);
00670      return err;
00671  }
00672  handle->rx_buffer_len = 0;
00673  handle->rx_buffer_pos = 0;
00674  handle->initialized = 1U;
00675  /* GCOVR_EXCL_BR_START LCOVR_EXCL_BR_START(assigned 1U on line above) */
00676  RX_ASSERT_POST(handle->initialized, "Handle initialization failed");
00677  /* GCOVR_EXCL_BR_STOP LCOVR_EXCL_BR_STOP */
00678  rx_log_debug(s_tag, "I2C comm initialized");
00679  return k_rx_ok;
00680 }

```

References [rx_i2c_comm_config_t::channel](#), [rx_i2c_comm_handle_t::channel_value](#), [rx_i2c_comm_handle_t::decoder](#), [rx_i2c_comm_config_t::device_addr](#), [rx_i2c_comm_handle_t::device_addr](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initialized](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), [rx_i2c_comm_handle_t::rx_buffer_pos](#), [s_tag](#), [s_zero_handle](#), [rx_i2c_comm_config_t::session](#), and [rx_i2c_comm_handle_t::session](#).

4.1.4.3 rx_i2c_comm_receive()

```

rx_err_t rx_i2c_comm_receive (
    rx_i2c_comm_handle_t * handle,
    rx_frame_t * frame,
    uint32_t timeout_ms) [nodiscard]

```

Receive and decode a frame from I2C controller (RPi5 write).

Polls ICSR2.RDRF for incoming bytes written by the I2C controller (RPi5), feeds them through the frame decoder, validates CRC-32, and extracts the payload. Non-blocking: returns `k_rx_err_no_data` if no complete frame is available.

Parameters

in,out	handle	Initialized handle
out	frame	Pointer to frame structure for output (must not be nullptr)
in	timeout_ms	Timeout in milliseconds (reserved for future use; poll is non-blocking)

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Complete frame received and decoded
<code>k_rx_err_invalid_arg</code>	handle or frame is nullptr
<code>k_rx_err_invalid_state</code>	Not initialized
<code>k_rx_err_no_data</code>	No complete frame available yet
<code>k_rx_err_crc_mismatch</code>	CRC-32 validation failed
<code>k_rx_err_protocol_error</code>	Invalid frame format

Precondition

handle initialized via [rx_i2c_comm_init\(\)](#)
 frame points to valid rx_frame_t

Postcondition

On success: frame contains decoded data, session RX sequence updated
 On k_rx_err_no_data: handle internal state unchanged, ready for next call

Note

Loop iterations bounded by k_i2c_comm_max_receive_iterations
 Thread safety: caller must provide external synchronization

See also

[rx_i2c_comm_send\(\)](#) Send frames to RPi5

Since

Version 1.0.0

Definition at line 966 of file [rx_i2c_comm.c](#).

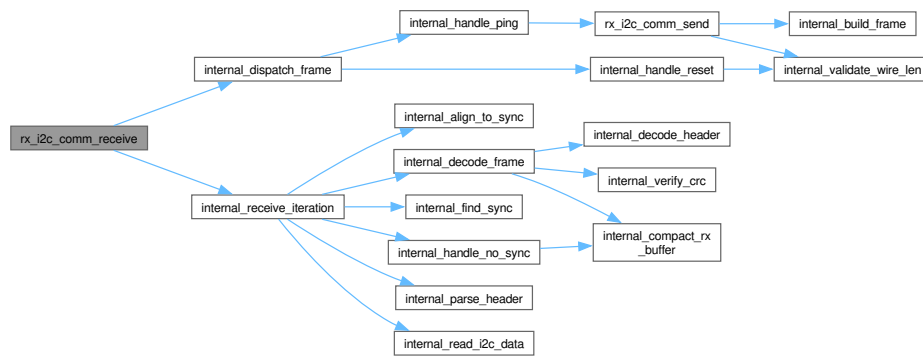
```

00967 {
00968     (void)timeout_ms; /* I2C receive is non-blocking; timeout reserved for future use */
00969
00970     if (handle == nullptr) {
00971         return k_rx_err_invalid_arg;
00972     }
00973
00974     if (frame == nullptr) {
00975         return k_rx_err_invalid_arg;
00976     }
00977
00978     if (!handle->initialized) {
00979         return k_rx_err_invalid_state;
00980     }
00981
00982     rx_err_t err = k_rx_ok;
00983
00984     for (uint32_t iterations = 0; iterations < k_i2c_comm_max_receive_iterations; iterations++) {
00985         rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
00986         if (result == k_receive_error) {
00987             return err;
00988         }
00989         if (result != k_receive_done) {
00990             continue;
00991         }
00992
00993         i2c_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
00994         if (dispatch == k_dispatch_error) {
00995             return err;
00996         }
00997         if (dispatch == k_dispatch_done) {
00998             return k_rx_ok;
00999         }
01000         /* k_dispatch_continue: loop for next frame */
01001     }
01002
01003     return k_rx_err_timeout;
01004 }

```

References [rx_i2c_comm_handle_t::initialized](#), [internal_dispatch_frame\(\)](#), [internal_receive_iteration\(\)](#), [k_dispatch_done](#), [k_dispatch_error](#), [k_i2c_comm_max_receive_iterations](#), [k_receive_done](#), and [k_receive_error](#).

Here is the call graph for this function:



4.1.4.4 rx_i2c_comm_send()

```

rx_err_t rx_i2c_comm_send (
    rx_i2c_comm_handle_t * handle,
    rx_frame_type_t type,
    uint8_t flags,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]

```

Send a frame over I2C (RIIC peripheral -> controller).

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation, then transmits it via `riic_peripheral_write()`. The RPi5 (I2C controller) must initiate a read transaction to receive data.

Parameters

in,out	handle	Initialized handle
in	type	Frame type (typically <code>k_frame_type_response</code>)
in	flags	Frame flags (ORed together, 0 for none)
in	payload	Pointer to payload data (may be nullptr if <code>payload_len == 0</code>)
in	payload_len	Payload length in bytes (0 to <code>k_frame_max_payload</code>)

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Frame queued for I2C controller read
<code>k_rx_err_invalid_arg</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	Not initialized
<code>k_rx_err_invalid_size</code>	Payload exceeds maximum frame size
<code>k_rx_err_timeout</code>	I2C transmit register not ready within timeout

Precondition

handle initialized via [rx_i2c_comm_init\(\)](#)

Postcondition

On success: session TX sequence incremented

Note

I2C send requires RPi5 to initiate a read transaction

Thread safety: caller must provide external synchronization

See also

[rx_i2c_comm_receive\(\)](#) Receive frames from RPi5

Since

Version 1.0.0

Definition at line 770 of file [rx_i2c_comm.c](#).

```

00775 {
00776     if (handle == nullptr) {
00777         return k_rx_err_invalid_arg;
00778     }
00779
00780     if (!handle->initialized) {
00781         rx_log_error(s_tag, "Handle not initialized");
00782         return k_rx_err_invalid_state;
00783     }
00784
00785     if (payload == nullptr && payload_len > 0) {
00786         return k_rx_err_invalid_arg;
00787     }
00788
00789     if (payload_len > k_frame_max_payload) {
00790         rx_log_error(s_tag, "Payload too large");
00791         return k_rx_err_invalid_size;
00792     }
00793
00794     uint16_t sequence = k_initial_sequence;
00795     rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00796     if (err != k_rx_ok) {
00797         rx_log_error(s_tag, "Session next_tx failed");
00798         return err;
00799     }
00800
00801     rx_frame_t frame = {};
00802     /* internal_build_frame only fails for invalid params; all params are validated above */
00803     (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00804
00805     uint32_t wire_len = 0;
00806     err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00807     if (err != k_rx_ok) {
00808         rx_log_error(s_tag, "Frame encode failed");
00809         return err;
00810     }
00811
00812     err = internal_validate_wire_len(wire_len);
00813     if (err != k_rx_ok) {
00814         return err;
00815     }
00816
00817     const riic_channel_t ch = {.value = handle->channel_value};
00818     err = riic_peripheral_write(ch, handle->tx_buffer, (uint16_t)wire_len);
00819     if (err != k_rx_ok) {
00820         rx_log_error(s_tag, "I2C peripheral write failed");
00821         return err;
00822     }
00823

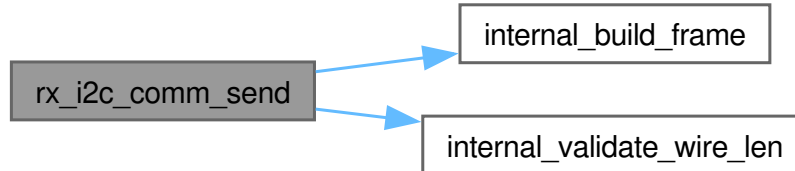
```

```
00824  return k_rx_ok;
00825 }
```

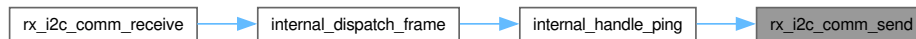
References [rx_i2c_comm_handle_t::channel_value](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initial_sequence](#), [internal_build_frame\(\)](#), [internal_validate_wire_len\(\)](#), [k_initial_sequence](#), [s_tag](#), [rx_i2c_comm_handle_t::session](#), and [rx_i2c_comm_handle_t::tx_buffer](#).

Referenced by [internal_handle_ping\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.2 rx_i2c_comm.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_i2c_comm.h
00003  * @brief High-Level I2C Peripheral Communication Layer for RX72N
00004  *
00005  * @details
00006  * Integrates the Frame Layer and RIIC0 (I2C) peripheral driver for reliable
00007  * communication with the RPi5 controller acting as I2C bus controller.
00008  * The RX72N acts as an I2C peripheral (device), responding to a fixed
00009  * 7-bit address set during initialization. This provides an alternative
00010  * communication channel when SPI or USB CDC are unavailable.
00011  *
00012  * @par System Architecture
00013  * @dot
00014  * digraph i2c_comm_architecture {
00015  *   rankdir=TB;
00016  *   node [shape=box, style=rounded];
00017  *
00018  *   subgraph cluster_rpi5 {
00019  *     label="Raspberry Pi 5 (I2C Controller)";
00020  *     style=filled;
00021  *     color=lightblue;
00022  *     i2c_dev [label="/dev/i2c-1\n(I2C bus controller)"];
00023  *     ros2 [label="ROS2 Node\nProtobuf Messages"];
00024  *   }
00025  *
00026  *   subgraph cluster_i2c {
00027  *     label="I2C Bus (SCL0/SDA0 - RIIC0)";
00028  *     style=filled;
00029  *     color=lightyellow;
00030  *     i2c_hw [label="RIIC0 Hardware\n100kHz/400kHz"];
00031  *   }
00032  *
00033  *   subgraph cluster_rx72n {
00034  *     label="RX72N Firmware (I2C Peripheral)";
00035  *     style=filled;
00036  *     color=lightgreen;
00037  *     i2c_comm [label="rx_i2c_comm\n(This Module)"];
00038  *     frame [label="rx_frame\nEncoder/Decoder"];
00039  *     nanopb [label="nanopb\nProtobuf"];
00040  *   }
00041  *
00042  *   ros2 -> i2c_dev [label="Protobuf"];
```

```

00043 * i2c_dev -> i2c_hw [label="I2C write/read"];
00044 * i2c_hw -> i2c_comm [label="RIIC0 peripheral"];
00045 * i2c_comm -> frame;
00046 * frame -> nanopb;
00047 * }
00048 * @enddot
00049 *
00050 * @par Peripheral Mode Operation
00051 * The RX72N acts as an I2C peripheral device at a configurable 7-bit address.
00052 * The RPi5 initiates all transactions as I2C controller. The RX72N uses
00053 * polling (ICSR2.RDRF for receive, ICSR2.TDRE for transmit) to handle
00054 * incoming writes and prepare outgoing read responses.
00055 *
00056 * @par Features
00057 * - Frame encoding/decoding with CRC-32 error detection
00058 * - Automatic sequence number tracking for reliable delivery
00059 * - Non-blocking receive operations
00060 * - Same wire protocol as USB/SPI/UART (interchangeable transport)
00061 * - Configurable I2C channel and device address
00062 *
00063 * @par Frame Protocol
00064 * Uses the same frame format as USB, SPI, and UART communication:
00065 * ```
00066 * +-----+-----+-----+-----+-----+-----+-----+
00067 * | SYNC | TYPE | FLAG | SEQ | LENGTH | PAYLOAD | CRC32 |
00068 * | 0xAA | 1B | 1B | 2B | 2B | N bytes | 4B |
00069 * +-----+-----+-----+-----+-----+-----+-----+
00070 * ```
00071 *
00072 * @par I2C Channel Assignment
00073 * | Channel | Peripheral | Addr | Purpose |
00074 * |-----|-----|-----|-----|
00075 * | RIIC0 | k_i2c_comm_default_channel | 0x42 (default) | Binary protocol (nanopb) |
00076 *
00077 * @par Performance Characteristics
00078 * | Metric | Value | Notes |
00079 * |-----|-----|-----|
00080 * | I2C speed | 400kHz max | Fast mode supported |
00081 * | Frame latency | ~1-5 ms | Depends on I2C clock speed |
00082 * | Max frame size | 2048 bytes | Buffer size limit |
00083 *
00084 * @par Memory Usage
00085 * | Component | Size | Description |
00086 * |-----|-----|-----|
00087 * | rx_i2c_comm_handle_t | ~4.2 KB | Per-instance handle |
00088 * | RX buffer | 2048 B | Staging buffer for receives |
00089 * | TX buffer | 2048 B | Staging buffer for sends |
00090 *
00091 * @par Thread Safety
00092 * The handle is NOT thread-safe. External synchronization (ThreadX mutex)
00093 * is required for concurrent access from multiple threads.
00094 *
00095 * @par Module Dependencies
00096 * - rx_frame.h: Frame encoding/decoding layer
00097 * - hardware.h: I2C HAL (riic_init_peripheral, riic_peripheral_read, riic_peripheral_write)
00098 * - rx_err.h: Error code definitions
00099 * - rx_session.h: Cross-transport sequence continuity
00100 *
00101 * @par NASA Power of 10 Compliance
00102 * - Rule 1: [PASS] No goto, setjmp, or recursion
00103 * - Rule 2: [PASS] Loop bounds via k_i2c_comm_max_receive_iterations
00104 * - Rule 3: [PASS] No dynamic memory (static buffers in handle)
00105 * - Rule 4: [PASS] Functions are concise
00106 * - Rule 5: [PASS] Minimum 2 pre/post conditions per public function
00107 * - Rule 6: [PASS] Variables declared at smallest scope
00108 * - Rule 7: [PASS] All return values checked
00109 * - Rule 8: [PASS] Constants use C23 typed enums
00110 * - Rule 10: [PASS] Compiles with -Wall -Wextra -Werror
00111 *
00112 * @par SOLID Principles
00113 * - S (SRP): I2C communication only, frame details in rx_frame
00114 * - O (OCP): Configurable channel/address via rx_i2c_comm_config_t
00115 * - L (LSP): Same protocol as rx_usb_comm / rx_uart_comm (interchangeable)
00116 * - I (ISP): Clean send/receive interface
00117 * - D (DIP): Depends on hardware.h HAL abstraction
00118 *
00119 * @see rx_frame.h Frame layer implementation
00120 * @see hardware.h I2C HAL
00121 * @see rx_usb_comm.h USB CDC communication (same protocol)
00122 * @see rx_uart_comm.h UART communication (same protocol)
00123 * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00124 *
00125 * @author Locked, Inc.
00126 * @date 2026-03-11
00127 * @copyright Copyright (c) 2026 Locked Inc.
00128 * SPDX-License-Identifier: MIT
00129 * @since Version 1.0.0

```

```

00130 */
00131
00132 #pragma once
00133
00134 #include <stdint.h>
00135
00136 #include "hardware.h"
00137 #include "rx_err.h"
00138 #include "rx_frame.h"
00139 #include "rx_session.h"
00140
00141 #ifdef __cplusplus
00142 extern "C" {
00143 #endif
00144
00145 /*
=====
00146 * Configuration Constants
00147 *
=====
00148 */
00149
00150 /**
00151  * @enum rx_i2c_comm_constants_t
00152  * @brief I2C communication configuration constants
00153  *
00154  * @details
00155  * Compile-time constants for I2C communication buffer sizes and loop bounds.
00156  * These values are tuned for the RX72N RIIC0 I2C implementation with typical
00157  * control message sizes.
00158  *
00159  * @par Buffer Sizing Rationale
00160  * - 2048 bytes accommodates largest expected protobuf message + frame overhead
00161  * - Provides headroom for frame header (8 bytes) and CRC (4 bytes)
00162  *
00163  * @invariant Buffer sizes must be >= maximum frame size
00164  * @invariant k_i2c_comm_max_receive_iterations bounds all receive loops
00165  *
00166  * @since Version 1.0.0
00167  */
00168 typedef enum : uint16_t {
00169 /**
00170  * @brief RX staging buffer size in bytes
00171  * @details
00172  * Buffer for accumulating incoming I2C data before frame decoding.
00173  * @par Value: 2048 bytes
00174  */
00175 k_i2c_comm_rx_buffer_size = 2048,
00176
00177 /**
00178  * @brief TX staging buffer size in bytes
00179  * @details
00180  * Buffer for assembled frames before I2C transmission.
00181  * @par Value: 2048 bytes
00182  */
00183 k_i2c_comm_tx_buffer_size = 2048,
00184
00185 /**
00186  * @brief Maximum iterations for receive loop (NASA Rule 2)
00187  * @details
00188  * Provides static upper bound for receive polling loops to ensure
00189  * bounded execution time even under pathological conditions.
00190  * @par Value: 1000 iterations
00191  */
00192 k_i2c_comm_max_receive_iterations = 1000,
00193 } rx_i2c_comm_constants_t;
00194
00195 /**
00196  * @enum rx_i2c_comm_defaults_t
00197  * @brief Default I2C channel and address for RPi5 communication
00198  *
00199  * @details
00200  * Default hardware channel (RIIC0) and 7-bit I2C device address (0x42) used
00201  * when the caller does not override via rx_i2c_comm_config_t.
00202  *
00203  * @par Address Selection
00204  * 0x42 ('B' in ASCII) is chosen to avoid conflicts with common I2C sensor
00205  * addresses (0x20-0x27 GPIO expanders, 0x3C/0x3D OLED, 0x48-0x4F ADC/temp,
00206  * 0x68/0x69 IMU, 0x76/0x77 barometer).
00207  *
00208  * @since Version 1.0.0
00209  */
00210 typedef enum : uint8_t {
00211 k_i2c_comm_default_channel = 0, /**< RIIC0 channel */
00212 k_i2c_comm_default_addr = 0x42, /**< Default 7-bit device address */
00213 } rx_i2c_comm_defaults_t;
00214

```

```

00215 /*
=====
00216 * Handle and Configuration
00217 *
=====
00218 */
00219
00220 /**
00221  * @struct rx_i2c_comm_handle_t
00222  * @brief I2C communication handle containing all operational state
00223  *
00224  * @details
00225  * Contains all state for I2C peripheral communication including frame
00226  * encoder/decoder, sequence tracking, and staging buffers. One handle
00227  * corresponds to one logical I2C communication channel (RIIC0).
00228  *
00229  * @par Memory Layout
00230  * | Offset | Size | Field | Description |
00231  * |-----|-----|-----|-----|
00232  * | 0 | ~50 B | encoder | Frame encoder state machine |
00233  * | 50 | ~50 B | decoder | Frame decoder state machine |
00234  * | 100 | 2048 B | rx_buffer | RX staging buffer |
00235  * | 2148 | 2048 B | tx_buffer | TX staging buffer |
00236  * | 4196 | 4 B | rx_buffer_len | Valid bytes in RX buffer |
00237  * | 4200 | 4 B | rx_buffer_pos | Current read position |
00238  * | 4204 | 4-8 B | session | Shared session state pointer |
00239  * | 4212 | 1 B | initialized | Init flag |
00240  * | 4213 | 1 B | channel_value | RIIC channel number |
00241  * | 4214 | 1 B | device_addr | 7-bit I2C device address |
00242  * | **Total** | **~4.2 KB** | | |
00243  *
00244  * @invariant initialized == 0 or initialized == 1
00245  * @invariant rx_buffer_pos <= rx_buffer_len
00246  * @invariant rx_buffer_len <= k_i2c_comm_rx_buffer_size
00247  *
00248  * @warning Do not access fields directly; use API functions
00249  * @attention Handle must be initialized via rx_i2c_comm_init() before use
00250  *
00251  * @see rx_i2c_comm_init() Initialize handle
00252  * @see rx_i2c_comm_deinit() Cleanup handle
00253  * @see rx_i2c_comm_config_t Configuration options
00254  *
00255  * @since Version 1.0.0
00256  */
00257 typedef struct {
00258     /**
00259      * @brief Frame encoder state machine
00260      * @details Encodes payloads into framed format with sync, header, and CRC.
00261      */
00262     rx_frame_encoder_t encoder;
00263
00264     /**
00265      * @brief Frame decoder state machine
00266      * @details Decodes incoming bytes, validates CRC, extracts payload.
00267      */
00268     rx_frame_decoder_t decoder;
00269
00270     /**
00271      * @brief RX staging buffer
00272      * @details Accumulates incoming I2C data before frame decoding.
00273      * @invariant Valid data in [0, rx_buffer_len)
00274      */
00275     uint8_t rx_buffer[k_i2c_comm_rx_buffer_size];
00276
00277     /**
00278      * @brief TX staging buffer
00279      * @details Holds assembled frame for I2C transmission.
00280      */
00281     uint8_t tx_buffer[k_i2c_comm_tx_buffer_size];
00282
00283     /**
00284      * @brief Number of valid bytes in RX buffer
00285      * @invariant rx_buffer_len <= k_i2c_comm_rx_buffer_size
00286      */
00287     uint32_t rx_buffer_len;
00288
00289     /**
00290      * @brief Current read position in RX buffer
00291      * @invariant rx_buffer_pos <= rx_buffer_len
00292      */
00293     uint32_t rx_buffer_pos;
00294
00295     /**
00296      * @brief Shared session state for cross-transport sequence continuity
00297      * @details Points to session state owned by rx_comm_manager.
00298      * @see rx_session_state_t
00299      */

```

```

00300 rx_session_state_t* session;
00301
00302 /**
00303  * @brief Initialization status flag
00304  * @par Values: 0 = not initialized, 1 = initialized
00305  */
00306 uint8_t initialized;
00307
00308 /**
00309  * @brief RIIC channel number
00310  * @details Stores the riic_channel_t.value configured during init.
00311  */
00312 uint8_t channel_value;
00313
00314 /**
00315  * @brief 7-bit I2C device address for this peripheral
00316  * @details Stores the i2c_device_addr_t.value configured during init.
00317  */
00318 uint8_t device_addr;
00319 } rx_i2c_comm_handle_t;
00320
00321 /**
00322  * @struct rx_i2c_comm_config_t
00323  * @brief I2C communication configuration options
00324  *
00325  * @details
00326  * Configuration passed to rx_i2c_comm_init(). The session pointer is
00327  * required for sequence tracking.
00328  *
00329  * @par Default Values
00330  * | Field      | Default                               | Description                               |
00331  * |-----|-----|-----|
00332  * | session    | (required)                           | Shared session state pointer            |
00333  * | channel    | k_i2c_comm_default_channel           | RIIC0 is the default I2C channel        |
00334  * | device_addr| k_i2c_comm_default_addr              | 0x42 default RPi5 peripheral address    |
00335  *
00336  * @see rx_i2c_comm_init() Uses this configuration
00337  *
00338  * @since Version 1.0.0
00339  */
00340 typedef struct {
00341     /**
00342      * @brief Shared session state for cross-transport sequence tracking
00343      * @details Must be initialized via rx_session_init() before passing here.
00344      * Required - must not be NULL.
00345      */
00346     rx_session_state_t* session;
00347
00348     /**
00349      * @brief RIIC channel to use
00350      * @details Selects which RIIC peripheral to use for I2C communication.
00351      * @par Default: {.value = k_i2c_comm_default_channel} (RIIC0)
00352      */
00353     riic_channel_t channel;
00354
00355     /**
00356      * @brief 7-bit I2C device address for this RX72N peripheral
00357      * @details RPi5 will address this device on the I2C bus.
00358      * @par Default: {.value = k_i2c_comm_default_addr} (0x42)
00359      */
00360     i2c_device_addr_t device_addr;
00361 } rx_i2c_comm_config_t;
00362
00363 /**
=====
00364  * Initialization
00365  *
=====
00366  */
00367
00368 /**
00369  * @brief Initialize I2C communication handle in peripheral mode
00370  *
00371  * @details
00372  * Sets up frame encoder/decoder, clears internal buffers, configures RIIC
00373  * peripheral at the specified I2C device address, and prepares the handle
00374  * for communication.
00375  *
00376  * @par Algorithm
00377  * 1. Validate handle pointer (NULL check)
00378  * 2. Clear handle memory
00379  * 3. Initialize frame encoder and decoder
00380  * 4. Copy configuration (channel, device_addr, session)
00381  * 5. Call riic_init_peripheral() to configure hardware
00382  * 6. Set initialized flag
00383  *
00384  * @param[out] handle Pointer to handle to initialize (must not be nullptr)

```

```

00385 * @param[in] config Configuration options (must not be NULL)
00386 *   - session: Required shared session state pointer
00387 *   - channel: RIIC channel (default RIIC0)
00388 *   - device_addr: 7-bit I2C device address
00389 *
00390 * @return rx_err_t Error code
00391 * @retval k_rx_ok Handle initialized successfully
00392 * @retval k_rx_err_invalid_arg handle or config is nullptr, or session is nullptr
00393 * @retval Other errors from riic_init_peripheral() or frame encoder/decoder init
00394 *
00395 * @pre handle points to allocated rx_i2c_comm_handle_t
00396 * @pre config->session points to initialized rx_session_state_t
00397 * @pre config->device_addr.value must be in range [0x08, 0x77]
00398 * @post handle->initialized == 1
00399 * @post RIIC peripheral will respond to config->device_addr on I2C bus
00400 * @post handle->encoder and handle->decoder are in their initial reset state
00401 *
00402 * @note Not thread-safe; call from single initialization context
00403 * @note Only initializes software handle; RIIC hardware configured via riic_init_peripheral()
00404 *
00405 * @see rx_i2c_comm_deinit() Cleanup resources
00406 * @see riic_channel_t I2C channel wrapper
00407 * @see i2c_device_addr_t I2C device address wrapper
00408 *
00409 * @since Version 1.0.0
00410 */
00411 [[nodiscard]] rx_err_t rx_i2c_comm_init(rx_i2c_comm_handle_t* handle,
00412                                         const rx_i2c_comm_config_t* config);
00413
00414 /**
00415 * @brief Deinitialize I2C communication handle
00416 *
00417 * @details
00418 * Clears the initialized flag to prevent further use of the handle.
00419 * Does not deinitialize the underlying RIIC hardware.
00420 *
00421 * @param[in,out] handle Pointer to initialized handle
00422 * @return k_rx_ok on success, k_rx_err_invalid_arg if handle is nullptr
00423 *
00424 * @pre handle must not be nullptr
00425 * @pre handle must have been initialized via rx_i2c_comm_init()
00426 * @post handle->initialized == 0
00427 * @post Further calls to rx_i2c_comm_send/receive will return k_rx_err_invalid_state
00428 *
00429 * @since Version 1.0.0
00430 */
00431 [[nodiscard]] rx_err_t rx_i2c_comm_deinit(rx_i2c_comm_handle_t* handle);
00432
00433 /*
=====
00434 * Send API
00435 *
=====
00436 */
00437 /**
00438 * @brief Send a frame over I2C (RIIC peripheral -> controller)
00439 *
00440 * @details
00441 * Encodes the payload into a frame with automatic sequence number assignment
00442 * and CRC-32 calculation, then transmits it via riic_peripheral_write().
00443 * The RPi5 (I2C controller) must initiate a read transaction to receive data.
00444 *
00445 * @param[in,out] handle Initialized handle
00446 * @param[in] type Frame type (typically k_frame_type_response)
00447 * @param[in] flags Frame flags (ORed together, 0 for none)
00448 * @param[in] payload Pointer to payload data (may be nullptr if payload_len == 0)
00449 * @param[in] payload_len Payload length in bytes (0 to k_frame_max_payload)
00450 *
00451 * @return rx_err_t Error code
00452 * @retval k_rx_ok Frame queued for I2C controller read
00453 * @retval k_rx_err_invalid_arg handle is nullptr
00454 * @retval k_rx_err_invalid_state Not initialized
00455 * @retval k_rx_err_invalid_size Payload exceeds maximum frame size
00456 * @retval k_rx_err_timeout I2C transmit register not ready within timeout
00457 *
00458 * @pre handle initialized via rx_i2c_comm_init()
00459 * @post On success: session TX sequence incremented
00460 *
00461 * @note I2C send requires RPi5 to initiate a read transaction
00462 * @note Thread safety: caller must provide external synchronization
00463 *
00464 * @see rx_i2c_comm_receive() Receive frames from RPi5
00465 *
00466 * @since Version 1.0.0
00467 */
00468 /*
00469 [[nodiscard]] rx_err_t rx_i2c_comm_send(rx_i2c_comm_handle_t* handle,

```

```

00470         rx_frame_type_t    type,
00471         uint8_t             flags,
00472         const uint8_t*      payload,
00473         uint32_t            payload_len);
00474
00475 /*
=====
00476 * Receive API
00477 *
=====
00478 */
00479
00480 /**
00481  * @brief Receive and decode a frame from I2C controller (RPi5 write)
00482  *
00483  * @details
00484  * Polls ICSR2.RDRF for incoming bytes written by the I2C controller (RPi5),
00485  * feeds them through the frame decoder, validates CRC-32, and extracts the
00486  * payload. Non-blocking: returns k_rx_err_no_data if no complete frame is
00487  * available.
00488  *
00489  * @param[in,out] handle Initialized handle
00490  * @param[out] frame Pointer to frame structure for output (must not be nullptr)
00491  * @param[in] timeout_ms Timeout in milliseconds (reserved for future use; poll is non-blocking)
00492  *
00493  * @return rx_err_t Error code
00494  * @retval k_rx_ok Complete frame received and decoded
00495  * @retval k_rx_err_invalid_arg handle or frame is nullptr
00496  * @retval k_rx_err_invalid_state Not initialized
00497  * @retval k_rx_err_no_data No complete frame available yet
00498  * @retval k_rx_err_crc_mismatch CRC-32 validation failed
00499  * @retval k_rx_err_protocol_error Invalid frame format
00500  *
00501  * @pre handle initialized via rx_i2c_comm_init()
00502  * @pre frame points to valid rx_frame_t
00503  * @post On success: frame contains decoded data, session RX sequence updated
00504  * @post On k_rx_err_no_data: handle internal state unchanged, ready for next call
00505  *
00506  * @note Loop iterations bounded by k_i2c_comm_max_receive_iterations
00507  * @note Thread safety: caller must provide external synchronization
00508  *
00509  * @see rx_i2c_comm_send() Send frames to RPi5
00510  *
00511  * @since Version 1.0.0
00512  */
00513 [[nodiscard]] rx_err_t
00514 rx_i2c_comm_receive(rx_i2c_comm_handle_t* handle, rx_frame_t* frame, uint32_t timeout_ms);
00515
00516 #ifdef __cplusplus
00517 }
00518 #endif

```

4.3 libs/rx_i2c_comm/src/rx_i2c_comm.c File Reference

High-Level I2C Peripheral Communication Layer Implementation.

```
#include "rx_i2c_comm.h"
```

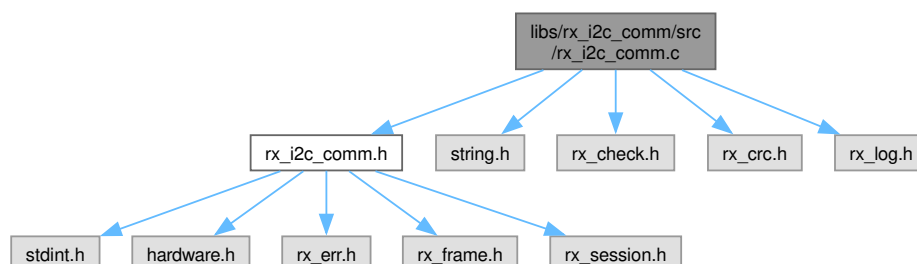
```
#include <string.h>
```

```
#include "rx_check.h"
```

```
#include "rx_crc.h"
```

```
#include "rx_log.h"
```

Include dependency graph for rx_i2c_comm.c:



Enumerations

- enum `rx_i2c_comm_header_size_t` : `uint8_t` { `k_frame_header_total` }
Total encoded frame header size in bytes.
- enum `i2c_crc32_seed_t` : `uint32_t` { `k_i2c_crc32_seed_initial` = 0U }
Initial seed value for CRC-32 output variable.
- enum `rx_i2c_comm_sequence_constants_t` : `uint16_t` { `k_initial_sequence` = 0 }
Sequence number initial value constants.
- enum `rx_i2c_comm_sync_constants_t` : `uint8_t` { `k_sync_second_byte_offset` = 1 }
Sync word search helper constants.
- enum `i2c_frame_header_offset_t` : `uint8_t` { `k_hdr_len_offset` = 4 }
Byte offsets into the raw frame header buffer.
- enum `rx_i2c_comm_sync_result_t` : `int32_t` { `k_sync_not_found` = -1 }
Sentinel value returned by `internal_find_sync()` when no sync is found.
- enum `rx_receive_result_t` : `uint8_t` { `k_receive_continue` = 0 , `k_receive_done` = 1 , `k_receive_error` = 2 }
Return codes for `internal_receive_iteration()`.
- enum `i2c_dispatch_result_t` : `uint8_t` { `k_dispatch_continue` = 0 , `k_dispatch_done` = 1 , `k_dispatch_error` = 2 }
Return codes for `internal_dispatch_frame()`.

Functions

- RX_STATIC_TESTABLE `rx_err_t internal_validate_wire_len` (`uint32_t` `wire_len`)
Validate that an encoded wire length fits within the I2C HAL transfer limit.
- RX_STATIC_TESTABLE `rx_err_t internal_decode_header` (`const uint8_t *data`, `const uint32_t data_len`, `rx_frame_t *frame`, `uint32_t *offset_out`)
Decode frame header from raw wire data.
- RX_STATIC_TESTABLE `rx_err_t internal_verify_crc` (`const uint8_t *data`, `uint32_t offset`, `uint32_t *crc_out`)
Verify CRC-32 IEEE checksum for received frame.
- RX_STATIC_TESTABLE `rx_err_t internal_read_i2c_data` (`rx_i2c_comm_handle_t *handle`)
Read available data from RIIC peripheral into staging buffer.
- RX_STATIC_TESTABLE `rx_err_t internal_compact_rx_buffer` (`rx_i2c_comm_handle_t *handle`)
Compact receive buffer by removing consumed data.
- RX_STATIC_TESTABLE `rx_err_t internal_find_sync` (`const rx_i2c_comm_handle_t *handle`, `int32_t *sync_pos`)
Search for frame sync word in receive buffer.
- RX_STATIC_TESTABLE `rx_err_t internal_handle_no_sync` (`rx_i2c_comm_handle_t *handle`)
Handle case when no sync word is found in buffer.
- RX_STATIC_TESTABLE `rx_err_t internal_decode_frame` (`rx_i2c_comm_handle_t *handle`, `rx_frame_t *frame`, `const uint32_t total_size`)
Decode a complete frame from buffer.
- RX_STATIC_TESTABLE `void internal_align_to_sync` (`rx_i2c_comm_handle_t *handle`, `const int32_t sync_pos`)
Align buffer position to sync word if found.
- RX_STATIC_TESTABLE `rx_err_t internal_parse_header` (`rx_i2c_comm_handle_t *handle`, `uint16_t *payload_len`, `uint32_t *total_size`)
Parse frame header and validate payload length.

- `RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration (rx_i2c_comm_handle_t *handle, rx_frame_t *frame, rx_err_t *err)`
Process one iteration of the frame receive loop.
- `rx_err_t rx_i2c_comm_init (rx_i2c_comm_handle_t *handle, const rx_i2c_comm_config_t *config)`
Initialize I2C communication handle in peripheral mode.
- `rx_err_t rx_i2c_comm_deinit (rx_i2c_comm_handle_t *handle)`
Deinitialize I2C communication handle.
- `RX_STATIC_TESTABLE rx_err_t internal_build_frame (rx_frame_t *frame, const uint16_t sequence, const rx_frame_type_t type, const uint8_t flags, const uint8_t *payload, const uint32_t payload_len)`
Build frame structure from individual parameters.
- `rx_err_t rx_i2c_comm_send (rx_i2c_comm_handle_t *handle, rx_frame_type_t type, uint8_t flags, const uint8_t *payload, uint32_t payload_len)`
Send a frame over I2C (RIIC peripheral -> controller).
- `RX_STATIC_TESTABLE rx_err_t internal_handle_ping (rx_i2c_comm_handle_t *handle, const rx_frame_t *frame)`
Handle a received PING frame by sending a PONG response.
- `RX_STATIC_TESTABLE rx_err_t internal_handle_reset (rx_i2c_comm_handle_t *handle)`
Handle a received RESET frame by sending RESET_ACK and resetting session.
- `static i2c_dispatch_result_t internal_dispatch_frame (rx_i2c_comm_handle_t *handle, rx_frame_t *frame, rx_err_t *err)`
Dispatch a decoded frame by type (PING/RESET/PONG/data).
- `rx_err_t rx_i2c_comm_receive (rx_i2c_comm_handle_t *handle, rx_frame_t *frame, uint32_t timeout_ms)`
Receive and decode a frame from I2C controller (RPi5 write).

Variables

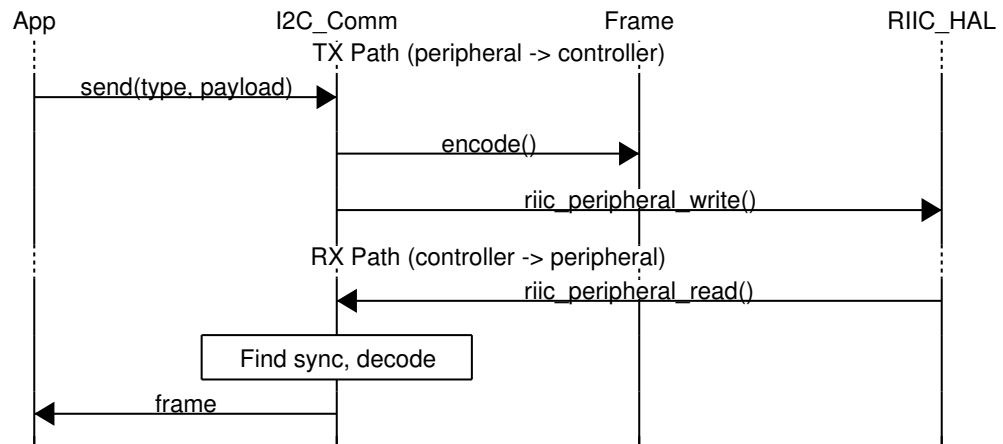
- `static const char *const s_tag = "I2C_COMM"`
- `static const rx_i2c_comm_handle_t s_zero_handle = {}`
Zero-initialized I2C comm handle template for stack-safe initialization.

4.3.1 Detailed Description

High-Level I2C Peripheral Communication Layer Implementation.

Implements reliable I2C communication by integrating the frame protocol layer (`rx_frame`) with the RIIC peripheral driver (`hardware.h` HAL). The RX72N acts as an I2C peripheral device, responding to address transactions from the RPi5 I2C controller. Handles frame encoding/decoding, CRC-32 validation, sequence number management, and receive buffer handling.

Implementation Architecture:



Channel Assignment:

Channel	Constant	Purpose
RIIC0	<code>k_i2c_comm_default_channel (0)</code>	Binary protocol

Performance:

Operation	Typical	Worst Case
Send (64B)	2 ms	10 ms
Receive	non-block	non-block

NASA Power of 10 Compliance:

- Rule 1: [OK] No goto, setjmp, recursion
- Rule 2: [OK] Loops bounded by `k_i2c_comm_max_receive_iterations`
- Rule 3: [OK] No dynamic allocation
- Rule 5: [OK] Pre/post conditions on all functions
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Typed enums for constants

SOLID Principles:

- SRP: Handles I2C framing only, delegates I/O to RIIC HAL
- DIP: Uses hardware.h HAL abstraction

See also

[rx_i2c_comm.h](#) Public API header
[rx_frame.h](#) Frame encoding layer
[hardware.h](#) RIIC HAL

Author

Locked, Inc.

Date

2026-03-11

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_i2c_comm.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 i2c_crc32_seed_t

enum [i2c_crc32_seed_t](#) : uint32_t

Initial seed value for CRC-32 output variable.

Used to zero-initialize the CRC output variable passed to [rx_crc32_ieee\(\)](#) before the calculation begins, avoiding use of a magic literal 0U.

Enumerator

k_i2c_crc32_seed_initial	Initial value for CRC-32 output variable
--	--

Definition at line 133 of file [rx_i2c_comm.c](#).

```
00133     : uint32_t {  
00134     k_i2c_crc32_seed_initial = 0U, /**< Initial value for CRC-32 output variable */  
00135 } i2c_crc32_seed_t;
```

4.3.2.2 i2c_dispatch_result_t

enum [i2c_dispatch_result_t](#) : uint8_t

Return codes for [internal_dispatch_frame\(\)](#).

Distinguishes between "frame consumed internally, continue loop", "frame dispatched to caller as data", and "error during dispatch".

Enumerator

k_dispatch_continue	Control frame consumed; continue receive loop
k_dispatch_done	Data frame validated; return to caller
k_dispatch_error	Error during dispatch; check err output

Definition at line 919 of file [rx_i2c_comm.c](#).

```
00919      : uint8_t {
00920  k_dispatch_continue = 0, /**< Control frame consumed; continue receive loop */
00921  k_dispatch_done     = 1, /**< Data frame validated; return to caller */
00922  k_dispatch_error     = 2, /**< Error during dispatch; check err output */
00923 } i2c_dispatch_result_t;
```

4.3.2.3 i2c_frame_header_offset_t

enum [i2c_frame_header_offset_t](#) : uint8_t

Byte offsets into the raw frame header buffer.

Provides named constants for the fixed field positions within the encoded frame header so that [internal_parse_header\(\)](#) avoids magic number indexing.

Enumerator

k_hdr_len_offset	Payload length field byte offset within header
------------------	--

Definition at line 275 of file [rx_i2c_comm.c](#).

```
00275      : uint8_t {
00276  k_hdr_len_offset = 4, /**< Payload length field byte offset within header */
00277 } i2c_frame_header_offset_t;
```

4.3.2.4 rx_i2c_comm_header_size_t

enum [rx_i2c_comm_header_size_t](#) : uint8_t

Total encoded frame header size in bytes.

Computed at compile time from the individual field size constants defined in [rx_frame.h](#). Used when determining how many bytes are needed to hold a complete frame header before the payload begins.

Enumerator

k_frame_header_total	Total header bytes
----------------------	--------------------

Definition at line 120 of file [rx_i2c_comm.c](#).

```
00120      : uint8_t {
00121  k_frame_header_total = k_frame_sync_size + k_frame_seq_size + k_frame_len_size +
00122                        k_frame_type_size + k_frame_flags_size, /**< Total header bytes */
00123 } rx_i2c_comm_header_size_t;
```

4.3.2.5 rx_i2c_comm_sequence_constants_t

```
enum rx_i2c_comm_sequence_constants_t : uint16_t
```

Sequence number initial value constants.

Provides a named constant for the sequence number starting value used when initializing a TX sequence or building a reset-ack frame, avoiding magic 0.

Enumerator

k_initial_sequence	Initial TX/RX sequence number
--------------------	-------------------------------

Definition at line 251 of file [rx_i2c_comm.c](#).

```
00251     : uint16_t {
00252   k_initial_sequence = 0, /**< Initial TX/RX sequence number */
00253 } rx_i2c_comm_sequence_constants_t;
```

4.3.2.6 rx_i2c_comm_sync_constants_t

```
enum rx_i2c_comm_sync_constants_t : uint8_t
```

Sync word search helper constants.

Provides a named offset for the second byte of the two-byte sync word when scanning the receive buffer for frame boundaries.

Enumerator

k_sync_second_byte_offset	Offset from first sync byte to second sync byte
---------------------------	---

Definition at line 263 of file [rx_i2c_comm.c](#).

```
00263     : uint8_t {
00264   k_sync_second_byte_offset = 1, /**< Offset from first sync byte to second sync byte */
00265 } rx_i2c_comm_sync_constants_t;
```

4.3.2.7 rx_i2c_comm_sync_result_t

```
enum rx_i2c_comm_sync_result_t : int32_t
```

Sentinel value returned by [internal_find_sync\(\)](#) when no sync is found.

Uses a signed int32_t so -1 is representable, distinguishing "not found" from any valid non-negative buffer position.

Enumerator

k_sync_not_found	Sync word not found in buffer
------------------	-------------------------------

Definition at line 292 of file [rx_i2c_comm.c](#).

```
00292     : int32_t {
00293   k_sync_not_found = -1, /**< Sync word not found in buffer */
00294 } rx_i2c_comm_sync_result_t;
```

4.3.2.8 rx_receive_result_t

enum [rx_receive_result_t](#) : uint8_t

Return codes for [internal_receive_iteration\(\)](#).

Allows the receive loop in [rx_i2c_comm_receive\(\)](#) to distinguish between "need more data", "frame ready", and "error occurred" without overloading the rx_err_t return type.

Enumerator

k_receive_continue	Continue to next iteration
k_receive_done	Frame received successfully
k_receive_error	Error occurred, check err output

Definition at line 305 of file [rx_i2c_comm.c](#).

```
00305      : uint8_t {
00306  k_receive_continue = 0, /**< Continue to next iteration */
00307  k_receive_done     = 1, /**< Frame received successfully */
00308  k_receive_error    = 2, /**< Error occurred, check err output */
00309 } rx_receive_result_t;
```

4.3.3 Function Documentation

4.3.3.1 internal_align_to_sync()

RX_STATIC_TESTABLE void [internal_align_to_sync](#) (
[rx_i2c_comm_handle_t](#) * handle,
 const int32_t sync_pos)

Align buffer position to sync word if found.

Advances rx_buffer_pos to sync_pos when sync_pos is ahead of the current position, discarding any bytes before the sync word.

Precondition

handle != nullptr
 sync_pos >= 0

Postcondition

handle->rx_buffer_pos >= (uint32_t)sync_pos
 No bytes before sync_pos remain accessible

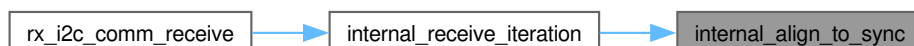
Definition at line 522 of file [rx_i2c_comm.c](#).

```
00523 {
00524   if ((uint32_t)sync_pos > handle->rx_buffer_pos) {
00525     handle->rx_buffer_pos = (uint32_t)sync_pos;
00526   }
00527 }
```

References [rx_i2c_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.2 internal_build_frame()

```
RX_STATIC_TESTABLE rx_err_t internal_build_frame (
    rx_frame_t * frame,
    const uint16_t sequence,
    const rx_frame_type_t type,
    const uint8_t flags,
    const uint8_t * payload,
    const uint32_t payload_len)
```

Build frame structure from individual parameters.

Populates all header fields of *frame and copies the payload bytes into frame->payload. Used by [rx_i2c_comm_send\(\)](#) to construct outgoing frames before encoding and transmission.

Precondition

```
frame != nullptr
payload != nullptr || payload_len == 0
```

Postcondition

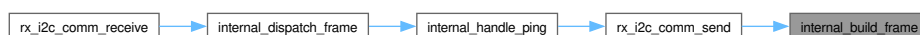
```
frame->header.sequence, type, length, flags are set
frame->payload[0..payload_len-1] copied from payload on k_rx_ok
```

Definition at line 727 of file [rx_i2c_comm.c](#).

```
00733 {
00734     RX_ASSERT(frame != nullptr, "Frame pointer is nullptr");
00735     if (frame == nullptr) {
00736         return k_rx_err_invalid_arg;
00737     }
00738     RX_ASSERT(payload != nullptr || payload_len == 0, "Payload nullptr but payload_len > 0");
00739     if (payload == nullptr && payload_len > 0) {
00740         return k_rx_err_invalid_arg;
00741     }
00742 }
00743
00744     if (payload_len > k_frame_max_payload) {
00745         return k_rx_err_invalid_size;
00746     }
00747
00748     frame->header.sequence = sequence;
00749     frame->header.length   = (uint16_t)payload_len;
00750     frame->header.type     = (uint8_t)type;
00751     frame->header.flags    = flags;
00752
00753     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START
00754      * gov -fprofile-arcs quirk with short-circuit && across multiple TUs --
00755      * test_i2c_internal_build_frame_with_payload calls this function with
00756      * payload != nullptr and payload_len = 4, asserts the 4-byte memcpy
00757      * happened, and passes. The `payload_len > 0 TRUE' sub-branch of the &&
00758      * still reports 0 hits in the merged .info. The behaviour is exercised;
00759      * only the counting is wrong. */
00760     if (payload != nullptr && payload_len > 0) {
00761         for (uint32_t i = 0; i < payload_len; i++) {
00762             frame->payload[i] = payload[i];
00763         }
00764     }
00765     /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00766     return k_rx_ok;
00767 }
00768 }
```

Referenced by [rx_i2c_comm_send\(\)](#).

Here is the caller graph for this function:



4.3.3.3 internal_compact_rx_buffer()

```
RX_STATIC_TESTABLE rx_err_t internal_compact_rx_buffer (
    rx_i2c_comm_handle_t * handle)
```

Compact receive buffer by removing consumed data.

Precondition

```
handle != nullptr
handle->rx_buffer_pos <= handle->rx_buffer_len
```

Postcondition

```
handle->rx_buffer_pos == 0
```

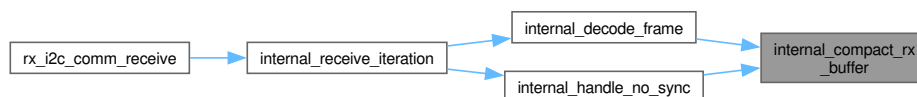
Definition at line 359 of file [rx_i2c_comm.c](#).

```
00360 {
00361     if (handle == nullptr) {
00362         return k_rx_err_invalid_arg;
00363     }
00364
00365     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00366         return k_rx_err_invalid_state;
00367     }
00368
00369     if (handle->rx_buffer_pos == 0) {
00370         return k_rx_ok;
00371     }
00372
00373     if (handle->rx_buffer_pos >= handle->rx_buffer_len) {
00374         handle->rx_buffer_len = 0;
00375         handle->rx_buffer_pos = 0;
00376     } else {
00377         const uint32_t remaining = handle->rx_buffer_len - handle->rx_buffer_pos;
00378         for (uint32_t i = 0; i < remaining; i++) {
00379             handle->rx_buffer[i] = handle->rx_buffer[handle->rx_buffer_pos + i];
00380         }
00381         handle->rx_buffer_len = remaining;
00382         handle->rx_buffer_pos = 0;
00383     }
00384
00385     return k_rx_ok;
00386 }
```

References [rx_i2c_comm_handle_t::rx_buffer](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), and [rx_i2c_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_decode_frame\(\)](#), and [internal_handle_no_sync\(\)](#).

Here is the caller graph for this function:



4.3.3.4 internal_decode_frame()

```
RX_STATIC_TESTABLE rx_err_t internal_decode_frame (
    rx_i2c_comm_handle_t * handle,
    rx_frame_t * frame,
    const uint32_t total_size)
```

Decode a complete frame from buffer.

Reads total_size bytes from the current buffer position, decodes the header, copies the payload, and verifies the CRC-32. Advances rx_buffer_pos by total_size regardless of success or failure, then compacts the buffer.

Precondition

handle != nullptr and initialized
 frame != nullptr
 total_size bytes available starting at handle->rx_buffer_pos

Postcondition

rx_buffer_pos advanced by total_size; buffer compacted
 *frame contains valid decoded frame on k_rx_ok

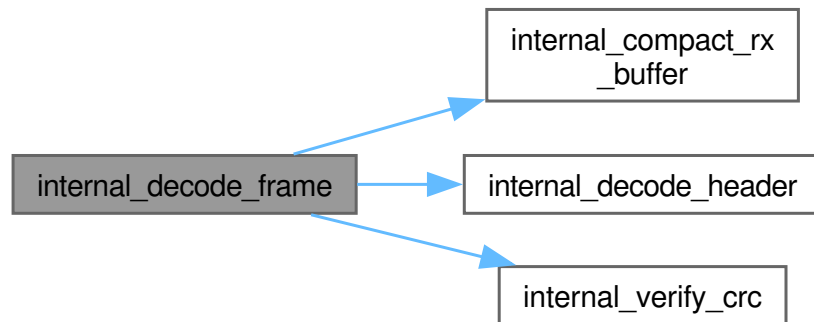
Definition at line 480 of file rx_i2c_comm.c.

```
00483 {
00484     const uint8_t* hdr    = handle->rx_buffer + handle->rx_buffer_pos;
00485     uint32_t      offset = 0;
00486
00487     /* internal_parse_header validates the header before this call;
00488      * internal_decode_header cannot fail for pre-validated data. */
00489     (void)internal_decode_header(hdr, total_size, frame, &offset);
00490
00491     if (frame->header.length > 0) {
00492         for (uint16_t i = 0; i < frame->header.length; i++) {
00493             frame->payload[i] = hdr[offset + i];
00494         }
00495         offset += frame->header.length;
00496     }
00497
00498     rx_err_t err = internal_verify_crc(hdr, offset, &frame->crc);
00499     if (err != k_rx_ok) {
00500         rx_log_error(s_tag, "Frame CRC check failed");
00501     }
00502
00503     handle->rx_buffer_pos += total_size;
00504     (void)internal_compact_rx_buffer(handle);
00505
00506     return err;
00507 }
```

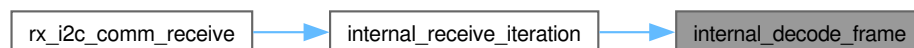
References [internal_compact_rx_buffer\(\)](#), [internal_decode_header\(\)](#), [internal_verify_crc\(\)](#), [rx_i2c_comm_handle_t::rx_i2c_comm_handle_t::rx_buffer_pos](#), and [s_tag](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.5 internal_decode_header()

```

RX_STATIC_TESTABLE rx_err_t internal_decode_header (
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame,
    uint32_t * offset_out)
  
```

Decode frame header from raw wire data.

Parses the fixed-size frame header from a raw byte buffer. Validates the sync word, extracts sequence number, payload length, type, and flags fields. Checks that the buffer is large enough for the declared payload before returning.

Precondition

```

data != nullptr
frame != nullptr
  
```

Postcondition

```

frame->header fields populated on success
*offset_out contains byte offset past header if not nullptr and k_rx_ok
  
```

Definition at line 152 of file [rx_i2c_comm.c](#).

```

00156 {
00157   RX_ASSERT(data != nullptr, "Data pointer is nullptr");
00158   RX_ASSERT(frame != nullptr, "Frame pointer is nullptr");
00159   if (data == nullptr || frame == nullptr) {
00160     return k_rx_err_invalid_arg;
00161   }
00162
00163   if (data_len < k_frame_min_size) {
00164     return k_rx_err_invalid_size;
00165   }
00166
00167   uint32_t offset = 0;
00168   const uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00169   if (sync_word != k_frame_sync_word) {
00170     return k_rx_err_protocol_error;
  
```

```

00171 }
00172 offset += k_frame_sync_size;
00173
00174 frame->header.sequence = rx_frame_read_le16(&data[offset]);
00175 offset += k_frame_seq_size;
00176
00177 frame->header.length = rx_frame_read_le16(&data[offset]);
00178 offset += k_frame_len_size;
00179
00180 if (frame->header.length > k_frame_max_payload) {
00181     return k_rx_err_invalid_size;
00182 }
00183
00184 frame->header.type = data[offset];
00185 offset += k_frame_type_size;
00186
00187 frame->header.flags = data[offset];
00188 offset += k_frame_flags_size;
00189
00190 if (offset_out != nullptr) {
00191     *offset_out = offset;
00192 }
00193
00194 return k_rx_ok;
00195 }

```

Referenced by [internal_decode_frame\(\)](#).

Here is the caller graph for this function:



4.3.3.6 internal_dispatch_frame()

```

i2c_dispatch_result_t internal_dispatch_frame (
    rx_i2c_comm_handle_t * handle,
    rx_frame_t * frame,
    rx_err_t * err) [static]

```

Dispatch a decoded frame by type (PING/RESET/PONG/data).

Precondition

handle != nullptr and initialized
 frame != nullptr with valid decoded header
 err != nullptr

Postcondition

On k_dispatch_continue: PONG/RESET_ACK may have been sent
 On k_dispatch_done: session RX sequence updated

Definition at line 937 of file [rx_i2c_comm.c](#).

```

00938 {
00939     /* PING: auto-send PONG, continue loop for next frame */
00940     if (frame->header.type == k_frame_type_ping) {
00941         *err = internal_handle_ping(handle, frame);
00942         return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00943     }
00944
00945     /* RESET: send RESET_ACK, reset session, continue loop */
00946     if (frame->header.type == k_frame_type_reset) {
00947         *err = internal_handle_reset(handle);

```

```

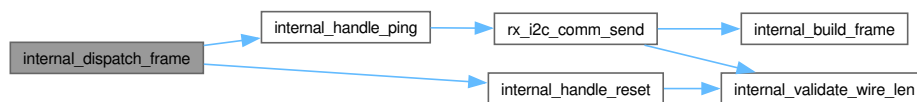
00948     return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00949 }
00950
00951 /* PONG / RESET_ACK: consume silently */
00952 if (frame->header.type == k_frame_type_pong || frame->header.type == k_frame_type_reset_ack) {
00953     return k_dispatch_continue;
00954 }
00955
00956 /* Data frame: validate sequence via session */
00957 rx_session_validate_result_t validate_result = k_session_validate_fail;
00958 *err = rx_session_validate_rx(handle->session, frame->header.sequence, &validate_result);
00959 if (*err != k_rx_ok) {
00960     rx_log_error(s_tag, "Session validate_rx returned error");
00961     return k_dispatch_error;
00962 }
00963 return k_dispatch_done;
00964 }

```

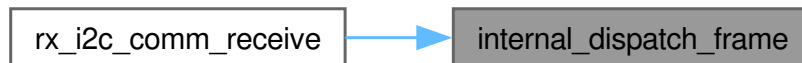
References [internal_handle_ping\(\)](#), [internal_handle_reset\(\)](#), [k_dispatch_continue](#), [k_dispatch_done](#), [k_dispatch_error](#), [s_tag](#), and [rx_i2c_comm_handle_t::session](#).

Referenced by [rx_i2c_comm_receive\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.7 internal_find_sync()

```

RX_STATIC_TESTABLE rx_err_t internal_find_sync (
    const rx_i2c_comm_handle_t * handle,
    int32_t * sync_pos)

```

Search for frame sync word in receive buffer.

Scans the receive buffer starting at `rx_buffer_pos` for the two-byte frame sync word (`k_frame_sync_word`) stored in little-endian byte order. Sets `*sync_pos` to the index of the matching byte or `k_sync_not_found` (-1).

Precondition

```

handle != nullptr
sync_pos != nullptr

```

Postcondition

*sync_pos contains the buffer index of sync word on k_rx_ok
 *sync_pos == k_sync_not_found on k_rx_err_not_found

Definition at line 403 of file [rx_i2c_comm.c](#).

```

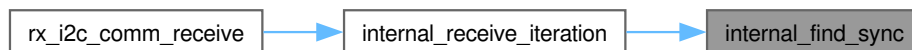
00405 {
00406     if (handle == nullptr) {
00407         return k_rx_err_invalid_arg;
00408     }
00409
00410     if (sync_pos == nullptr) {
00411         return k_rx_err_invalid_arg;
00412     }
00413
00414     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00415         return k_rx_err_invalid_state;
00416     }
00417
00418     const uint8_t sync_low = (uint8_t)(k_frame_sync_word & k_rx_byte_mask);
00419     const uint8_t sync_high = (uint8_t)(k_frame_sync_word » k_rx_le16_high_shift);
00420
00421     for (uint32_t i = handle->rx_buffer_pos; i + k_sync_second_byte_offset < handle->rx_buffer_len;
00422          i++) {
00423         if (handle->rx_buffer[i] == sync_low &&
00424             handle->rx_buffer[i + k_sync_second_byte_offset] == sync_high) {
00425             *sync_pos = (int32_t)i;
00426             return k_rx_ok;
00427         }
00428     }
00429
00430     *sync_pos = k_sync_not_found;
00431     return k_rx_err_not_found;
00432 }

```

References [k_sync_not_found](#), [k_sync_second_byte_offset](#), [rx_i2c_comm_handle_t::rx_buffer](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), and [rx_i2c_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.8 internal_handle_no_sync()

```

RX_STATIC_TESTABLE rx_err_t internal_handle_no_sync (
    rx_i2c_comm_handle_t * handle)

```

Handle case when no sync word is found in buffer.

When no sync word is present, advances the buffer position by one byte if the buffer is full (to make room for new data), then compacts the buffer by discarding consumed bytes from the front.

Precondition

handle != nullptr
 handle->rx_buffer_pos <= handle->rx_buffer_len

Postcondition

Buffer compacted; rx_buffer_pos == 0 on success

No sync word present in remaining buffer data

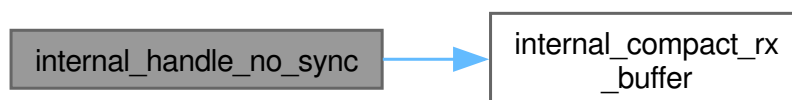
Definition at line 449 of file [rx_i2c_comm.c](#).

```
00450 {
00451     if (handle == nullptr) {
00452         return k_rx_err_invalid_arg;
00453     }
00454
00455     if (handle->rx_buffer_len >= k_i2c_comm_rx_buffer_size) {
00456         handle->rx_buffer_pos++;
00457     }
00458
00459     (void)internal_compact_rx_buffer(handle);
00460
00461     return k_rx_ok;
00462 }
```

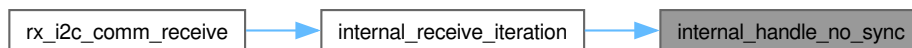
References [internal_compact_rx_buffer\(\)](#), [k_i2c_comm_rx_buffer_size](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), and [rx_i2c_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.9 internal_handle_ping()

```
RX_STATIC_TESTABLE rx_err_t internal_handle_ping (
    rx_i2c_comm_handle_t * handle,
    const rx_frame_t * frame)
```

Handle a received PING frame by sending a PONG response.

Sends a PONG frame echoing the received PING payload back to the I2C controller. Called automatically by [rx_i2c_comm_receive\(\)](#) when a PING frame is decoded.

Precondition

handle != nullptr and handle->initialized == true

frame != nullptr and frame->header.type == k_frame_type_ping

Postcondition

PONG frame transmitted over I2C peripheral write
 Session TX sequence number incremented

Definition at line 847 of file [rx_i2c_comm.c](#).

```

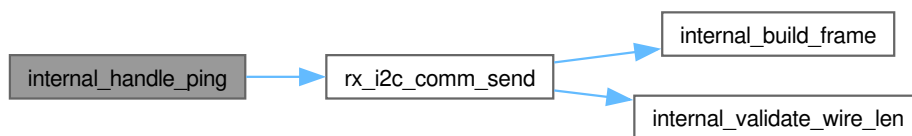
00849 {
00850     rx_err_t pong_err = rx_i2c_comm_send(handle,
00851                                         k_frame_type_pong,
00852                                         k_frame_flag_none,
00853                                         frame->payload,
00854                                         frame->header.length);
00855     if (pong_err != k_rx_ok) {
00856         rx_log_error(s_tag, "Failed to send PONG response");
00857         return pong_err;
00858     }
00859     rx_log_debug(s_tag, "Auto-responded with PONG");
00860     return k_rx_ok;
00861 }

```

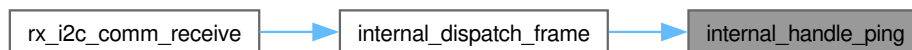
References [rx_i2c_comm_send\(\)](#), and [s_tag](#).

Referenced by [internal_dispatch_frame\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.10 internal_handle_reset()

```

RX_STATIC_TESTABLE rx_err_t internal_handle_reset (
    rx_i2c_comm_handle_t * handle)

```

Handle a received RESET frame by sending RESET_ACK and resetting session.

Constructs and transmits a RESET_ACK frame over the I2C peripheral write channel, then resets the session state machine. Called automatically by [rx_i2c_comm_receive\(\)](#) when a RESET frame is decoded.

Precondition

handle != nullptr and handle->initialized == true
 handle->session != nullptr and initialized

Postcondition

RESET_ACK frame transmitted over I2C peripheral write
handle->session reset to initial state on k_rx_ok

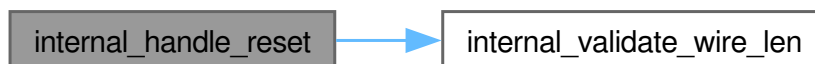
Definition at line 878 of file [rx_i2c_comm.c](#).

```
00879 {
00880     uint16_t reset_ack_seq = k_initial_sequence;
00881     const rx_err_t seq_err = rx_session_next_tx(handle->session, &reset_ack_seq);
00882     if (seq_err != k_rx_ok) {
00883         return seq_err;
00884     }
00885     rx_frame_t reset_ack_frame = {};
00886     /* rx_frame_create_reset_ack only fails for nullptr; reset_ack_frame is on the stack */
00887     (void)rx_frame_create_reset_ack(&reset_ack_frame, reset_ack_seq);
00888     uint32_t wire_len = 0;
00889     /* rx_frame_encode only fails for nullptr/uninitialized args; all are valid here */
00890     (void)rx_frame_encode(&handle->encoder, &reset_ack_frame, handle->tx_buffer, &wire_len);
00891     /* Reset ACK frame is always within I2C transfer limits (fixed small size) */
00892     (void)internal_validate_wire_len(wire_len);
00893     const riic_channel_t ch = {.value = handle->channel_value};
00894     rx_err_t write_err = riic_peripheral_write(ch, handle->tx_buffer, (uint16_t)wire_len);
00895     if (write_err != k_rx_ok) {
00896         rx_log_error(s_tag, "Failed to send RESET_ACK");
00897         return write_err;
00898     }
00899     rx_log_debug(s_tag, "Auto-responded with RESET_ACK");
00900     /* rx_session_reset only fails for nullptr; handle->session is validated by caller */
00901     (void)rx_session_reset(handle->session);
00902     return k_rx_ok;
00903 }
```

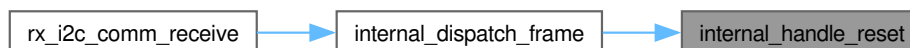
References [rx_i2c_comm_handle_t::channel_value](#), [rx_i2c_comm_handle_t::encoder](#), [internal_validate_wire_len\(\)](#), [k_initial_sequence](#), [s_tag](#), [rx_i2c_comm_handle_t::session](#), and [rx_i2c_comm_handle_t::tx_buffer](#).

Referenced by [internal_dispatch_frame\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.11 internal_parse_header()

```
RX_STATIC_TESTABLE rx_err_t internal_parse_header (
    rx_i2c_comm_handle_t * handle,
    uint16_t * payload_len,
    uint32_t * total_size)
```

Parse frame header and validate payload length.

Reads the payload length field from the frame header at the current buffer position and validates it against `k_frame_max_payload`. Computes the total expected frame size if the length is valid, or advances the buffer position past the sync word if invalid.

Precondition

handle != nullptr with at least k_frame_header_total bytes available
 payload_len != nullptr
 total_size != nullptr

Postcondition

On k_rx_ok: *total_size == k_frame_header_total + *payload_len + k_frame_crc_size
 On k_rx_err_invalid_size: rx_buffer_pos advanced by k_frame_sync_size

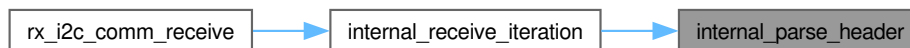
Definition at line 546 of file rx_i2c_comm.c.

```
00549 {
00550     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00551     *payload_len      = rx_frame_read_le16(&hdr[k_hdr_len_offset]);
00552
00553     if (*payload_len > k_frame_max_payload) {
00554         rx_log_warn(s_tag, "Invalid payload length, skipping");
00555         handle->rx_buffer_pos += k_frame_sync_size;
00556         return k_rx_err_invalid_size;
00557     }
00558
00559     *total_size = k_frame_header_total + *payload_len + k_frame_crc_size;
00560     return k_rx_ok;
00561 }
```

References [k_frame_header_total](#), [k_hdr_len_offset](#), [rx_i2c_comm_handle_t::rx_buffer](#), [rx_i2c_comm_handle_t::rx_buffer_pos](#), and [s_tag](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.12 internal_read_i2c_data()

```
RX_STATIC_TESTABLE rx_err_t internal_read_i2c_data (
    rx_i2c_comm_handle_t * handle)
```

Read available data from RIIC peripheral into staging buffer.

Precondition

handle != nullptr
 handle->rx_buffer_len <= k_i2c_comm_rx_buffer_size

Postcondition

handle->rx_buffer_len <= k_i2c_comm_rx_buffer_size

Definition at line 320 of file [rx_i2c_comm.c](#).

```

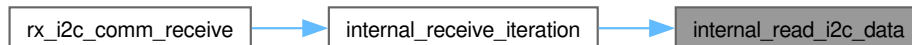
00321 {
00322     if (handle == nullptr) {
00323         return k_rx_err_invalid_arg;
00324     }
00325
00326     const uint32_t space = k_i2c_comm_rx_buffer_size - handle->rx_buffer_len;
00327     if (space == 0) {
00328         return k_rx_ok;
00329     }
00330
00331     /* Clamp to the HAL transfer limit so riic_peripheral_read never receives
00332      * a length that exceeds k_riic_peripheral_transfer_limit (256 bytes). */
00333     const uint16_t read_len = (space > (uint32_t)k_riic_peripheral_transfer_limit)
00334         ? k_riic_peripheral_transfer_limit
00335         : (uint16_t)space;
00336
00337     const riic_channel_t ch = {.value = handle->channel_value};
00338     uint16_t bytes_read = 0;
00339     const rx_err_t err =
00340         riic_peripheral_read(ch, handle->rx_buffer + handle->rx_buffer_len, read_len, &bytes_read);
00341     if (err != k_rx_ok) {
00342         return err;
00343     }
00344
00345     handle->rx_buffer_len += (uint32_t)bytes_read;
00346
00347     return k_rx_ok;
00348 }

```

References [rx_i2c_comm_handle_t::channel_value](#), [k_i2c_comm_rx_buffer_size](#), [rx_i2c_comm_handle_t::rx_buffer](#) and [rx_i2c_comm_handle_t::rx_buffer_len](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.13 internal_receive_iteration()

```

RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration (
    rx_i2c_comm_handle_t * handle,
    rx_frame_t * frame,
    rx_err_t * err)

```

Process one iteration of the frame receive loop.

Performs a single pass of the receive state machine: reads available I2C data, searches for a sync word, validates the header, and decodes a complete frame when enough bytes are present.

Precondition

handle != nullptr and handle->initialized == true
 frame != nullptr
 err != nullptr

Postcondition

On `k_receive_done`: `*frame` contains a valid, CRC-verified frame

On `k_receive_error`: `*err` contains the failure code

Definition at line 579 of file [rx_i2c_comm.c](#).

```

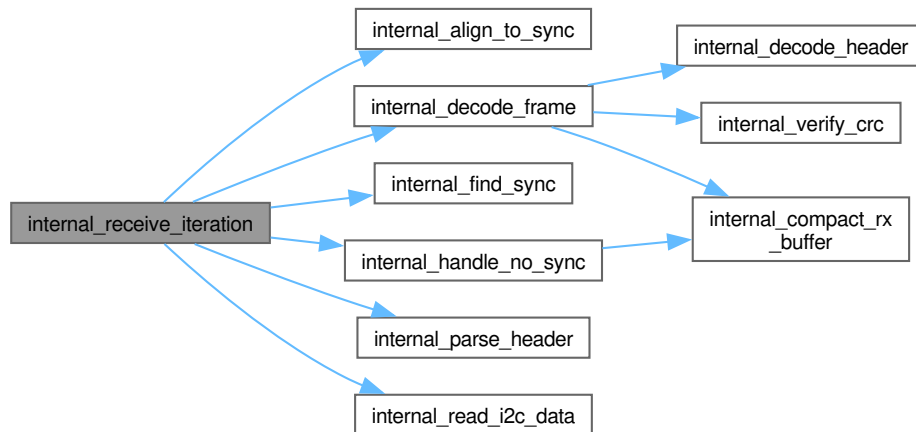
00582 {
00583     *err = internal_read_i2c_data(handle);
00584     /* k_rx_err_timeout means the HAL had no new bytes this iteration; treat as
00585      * "buffer unchanged" and fall through to the sync-search so the caller can
00586      * re-attempt up to k_i2c_comm_max_receive_iterations times before giving up. */
00587     if (*err != k_rx_ok && *err != k_rx_err_timeout) {
00588         return k_receive_error;
00589     }
00590
00591     if (handle->rx_buffer_len == handle->rx_buffer_pos) {
00592         *err = k_rx_err_no_data;
00593         return k_receive_error;
00594     }
00595
00596     int32_t sync_pos = k_sync_not_found;
00597     *err = internal_find_sync(handle, &sync_pos);
00598     if (*err == k_rx_err_not_found) {
00599         (void)internal_handle_no_sync(handle);
00600         *err = k_rx_err_no_data;
00601         return k_receive_error;
00602     }
00603
00604     internal_align_to_sync(handle, sync_pos);
00605
00606     const uint32_t available = handle->rx_buffer_len - handle->rx_buffer_pos;
00607     if (available < k_frame_header_total) {
00608         *err = k_rx_err_no_data;
00609         return k_receive_error;
00610     }
00611
00612     uint32_t total_size = 0;
00613     uint16_t payload_len = 0;
00614     *err = internal_parse_header(handle, &payload_len, &total_size);
00615     if (*err == k_rx_err_invalid_size) {
00616         /* Invalid payload length; buffer advanced, continue to find next sync */
00617         *err = k_rx_ok;
00618         return k_receive_continue;
00619     }
00620
00621     if (available < total_size) {
00622         *err = k_rx_err_no_data;
00623         return k_receive_error;
00624     }
00625
00626     *err = internal_decode_frame(handle, frame, total_size);
00627     return (*err != k_rx_ok) ? k_receive_error : k_receive_done;
00628 }

```

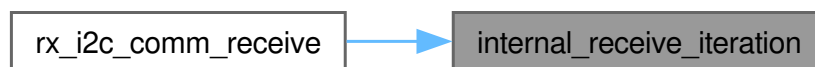
References [internal_align_to_sync\(\)](#), [internal_decode_frame\(\)](#), [internal_find_sync\(\)](#), [internal_handle_no_sync\(\)](#), [internal_parse_header\(\)](#), [internal_read_i2c_data\(\)](#), [k_frame_header_total](#), [k_receive_continue](#), [k_receive_done](#), [k_receive_error](#), [k_sync_not_found](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), and [rx_i2c_comm_handle_t::rx_buffer_pos](#).

Referenced by [rx_i2c_comm_receive\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.14 internal_validate_wire_len()

```

RX_STATIC_TESTABLE rx_err_t internal_validate_wire_len (
    uint32_t wire_len)

```

Validate that an encoded wire length fits within the I2C HAL transfer limit.

Checks `wire_len` against `k_riic_peripheral_transfer_limit` and logs an error if the limit is exceeded. Called before every `riic_peripheral_write` to prevent an unchecked narrowing cast from `uint32_t` to `uint16_t`.

Precondition

`wire_len` reflects the output of `rx_frame_encode()`
`k_riic_peripheral_transfer_limit` is defined in `rx_riic.h`

Postcondition

Error logged if limit exceeded
 Caller must propagate `k_rx_err_invalid_size` without calling `riic_peripheral_write`

Since

Version 1.0.0

Definition at line 95 of file [rx_i2c_comm.c](#).

```

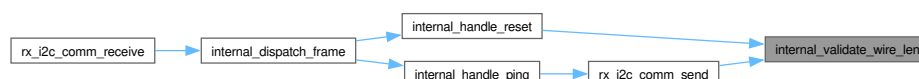
00096 {
00097     if (wire_len > (uint32_t)k_riic_peripheral_transfer_limit) {
00098         rx_log_error(s_tag, "Encoded frame exceeds I2C HAL transfer limit; cannot send");
00099         return k_rx_err_invalid_size;
00100     }
00101     return k_rx_ok;
00102 }

```

References [s_tag](#).

Referenced by [internal_handle_reset\(\)](#), and [rx_i2c_comm_send\(\)](#).

Here is the caller graph for this function:



4.3.3.15 internal_verify_crc()

```
RX_STATIC_TESTABLE rx_err_t internal_verify_crc (
    const uint8_t * data,
    uint32_t offset,
    uint32_t * crc_out)
```

Verify CRC-32 IEEE checksum for received frame.

Reads the 4-byte CRC stored at data[offset..offset+3] (little-endian), computes CRC-32 IEEE 802.3 over data[0..offset-1], and compares the two values to confirm frame integrity.

Precondition

```
data != nullptr
offset >= (k_frame_min_size - k_frame_crc_size)
```

Postcondition

```
Frame integrity validated on k_rx_ok return
*crc_out contains verified CRC value if not nullptr and k_rx_ok
```

Definition at line 212 of file [rx_i2c_comm.c](#).

```
00215 {
00216     RX_ASSERT(data != nullptr, "Data pointer is nullptr");
00217     if (data == nullptr) {
00218         return k_rx_err_invalid_arg;
00219     }
00220
00221     RX_ASSERT(offset >= (k_frame_min_size - k_frame_crc_size),
00222         "CRC offset too small for valid frame");
00223     if (offset < (k_frame_min_size - k_frame_crc_size)) {
00224         return k_rx_err_invalid_arg;
00225     }
00226
00227     const uint32_t received_crc = rx_frame_read_le32(&data[offset]);
00228     uint32_t calculated_crc = k_i2c_crc32_seed_initial;
00229     /* rx_crc32_ieee only fails on null/zero-len; data and offset are validated by caller */
00230     (void)rx_crc32_ieee(data, offset, &calculated_crc);
00231
00232     if (received_crc != calculated_crc) {
00233         return k_rx_err_crc_mismatch;
00234     }
00235
00236     if (crc_out != nullptr) {
00237         *crc_out = received_crc;
00238     }
00239
00240     return k_rx_ok;
00241 }
```

References [k_i2c_crc32_seed_initial](#).

Referenced by [internal_decode_frame\(\)](#).

Here is the caller graph for this function:



4.3.3.16 rx_i2c_comm_deinit()

```
rx_err_t rx_i2c_comm_deinit (
    rx_i2c_comm_handle_t * handle) [nodiscard]
```

Deinitialize I2C communication handle.

Clears the initialized flag to prevent further use of the handle. Does not deinitialize the underlying RIIC hardware.

Parameters

in,out	handle	Pointer to initialized handle
--------	--------	-------------------------------

Returns

k_rx_ok on success, k_rx_err_invalid_arg if handle is nullptr

Precondition

handle must not be nullptr

handle must have been initialized via [rx_i2c_comm_init\(\)](#)

Postcondition

handle->initialized == 0

Further calls to rx_i2c_comm_send/receive will return k_rx_err_invalid_state

Since

Version 1.0.0

Definition at line 685 of file [rx_i2c_comm.c](#).

```
00686 {
00687     if (handle == nullptr) {
00688         return k_rx_err_invalid_arg;
00689     }
00690
00691     RX_ASSERT_PRE(handle->initialized, "Attempt to deinitialize uninitialized I2C comm handle");
00692
00693     (void)rx_frame_encoder_deinit(&handle->encoder);
00694     (void)rx_frame_decoder_deinit(&handle->decoder);
00695
00696     /* Hardware (RIIC peripheral) cleanup is the caller's responsibility.
00697      * The hardware.h HAL does not expose a riic_deinit() function; the RIIC
00698      * module stop bit (MSTPCRB) must be set by the caller if power-gating is
00699      * needed after deinitializing this comm layer. */
00700
00701     handle->initialized = 0U;
00702
00703     rx_log_debug(s_tag, "I2C comm deinitialized");
00704     return k_rx_ok;
00705 }
```

References [rx_i2c_comm_handle_t::decoder](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initialized](#), and [s_tag](#).

4.3.3.17 rx_i2c_comm_init()

```
rx_err_t rx_i2c_comm_init (
    rx_i2c_comm_handle_t * handle,
    const rx_i2c_comm_config_t * config) [nodiscard]
```

Initialize I2C communication handle in peripheral mode.

Sets up frame encoder/decoder, clears internal buffers, configures RIIC peripheral at the specified I2C device address, and prepares the handle for communication.

Algorithm

1. Validate handle pointer (NULL check)
2. Clear handle memory
3. Initialize frame encoder and decoder
4. Copy configuration (channel, device_addr, session)
5. Call riic_init_peripheral() to configure hardware
6. Set initialized flag

Parameters

out	handle	Pointer to handle to initialize (must not be nullptr)
in	config	Configuration options (must not be NULL) • session: Required shared session state pointer • channel: RIIC channel (default RIIC0) • device_addr: 7-bit I2C device address

Returns

rx_err_t Error code

Return values

k_rx_ok	Handle initialized successfully
k_rx_err_invalid_arg	handle or config is nullptr, or session is nullptr
Other	errors from riic_init_peripheral() or frame encoder/decoder init

Precondition

handle points to allocated [rx_i2c_comm_handle_t](#)
 config->session points to initialized [rx_session_state_t](#)
 config->device_addr.value must be in range [0x08, 0x77]

Postcondition

handle->initialized == 1

RIIC peripheral will respond to config->device_addr on I2C bus

handle->encoder and handle->decoder are in their initial reset state

Note

Not thread-safe; call from single initialization context

Only initializes software handle; RIIC hardware configured via riic_init_peripheral()

See also

[rx_i2c_comm_deinit\(\)](#) Cleanup resources

[riic_channel_t](#) I2C channel wrapper

[i2c_device_addr_t](#) I2C device address wrapper

Since

Version 1.0.0

Definition at line 645 of file [rx_i2c_comm.c](#).

```

00646 {
00647     if (handle == nullptr) {
00648         return k_rx_err_invalid_arg;
00649     }
00650
00651     *handle = s_zero_handle;
00652
00653     if (config == nullptr || config->session == nullptr) {
00654         return k_rx_err_invalid_arg;
00655     }
00656     handle->session = config->session;
00657     handle->channel_value = config->channel.value;
00658     handle->device_addr = config->device_addr.value;
00659
00660     /* encoder/decoder init only fail for nullptr, which is impossible since handle is non-null */
00661     (void)rx_frame_encoder_init(&handle->encoder);
00662     (void)rx_frame_decoder_init(&handle->decoder);
00663
00664     /* Initialize RIIC in peripheral mode */
00665     rx_err_t err = riic_init_peripheral(config->channel, config->device_addr);
00666     if (err != k_rx_ok) {
00667         rx_log_error(s_tag, "Failed to init RIIC peripheral mode");
00668
00669         (void)rx_frame_encoder_deinit(&handle->encoder);
00670         (void)rx_frame_decoder_deinit(&handle->decoder);
00671         return err;
00672     }
00673
00674     handle->rx_buffer_len = 0;
00675     handle->rx_buffer_pos = 0;
00676     handle->initialized = 1U;
00677     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(assigned 1U on line above) */
00678     RX_ASSERT_POST(handle->initialized, "Handle initialization failed");
00679     /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00680
00681     rx_log_debug(s_tag, "I2C comm initialized");
00682     return k_rx_ok;
00683 }

```

References [rx_i2c_comm_config_t::channel](#), [rx_i2c_comm_handle_t::channel_value](#), [rx_i2c_comm_handle_t::decoder](#), [rx_i2c_comm_config_t::device_addr](#), [rx_i2c_comm_handle_t::device_addr](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initialized](#), [rx_i2c_comm_handle_t::rx_buffer_len](#), [rx_i2c_comm_handle_t::rx_buffer_pos](#), [s_tag](#), [s_zero_handle](#), [rx_i2c_comm_config_t::session](#), and [rx_i2c_comm_handle_t::session](#).

4.3.3.18 rx_i2c_comm_receive()

```
rx_err_t rx_i2c_comm_receive (
    rx_i2c_comm_handle_t * handle,
    rx_frame_t * frame,
    uint32_t timeout_ms) [nodiscard]
```

Receive and decode a frame from I2C controller (RPi5 write).

Polls ICSR2.RDRF for incoming bytes written by the I2C controller (RPi5), feeds them through the frame decoder, validates CRC-32, and extracts the payload. Non-blocking: returns k_rx_err_no_data if no complete frame is available.

Parameters

in,out	handle	Initialized handle
out	frame	Pointer to frame structure for output (must not be nullptr)
in	timeout_ms	Timeout in milliseconds (reserved for future use; poll is non-blocking)

Returns

rx_err_t Error code

Return values

k_rx_ok	Complete frame received and decoded
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	Not initialized
k_rx_err_no_data	No complete frame available yet
k_rx_err_crc_mismatch	CRC-32 validation failed
k_rx_err_protocol_error	Invalid frame format

Precondition

handle initialized via rx_i2c_comm_init()
 frame points to valid rx_frame_t

Postcondition

On success: frame contains decoded data, session RX sequence updated
 On k_rx_err_no_data: handle internal state unchanged, ready for next call

Note

Loop iterations bounded by k_i2c_comm_max_receive_iterations
 Thread safety: caller must provide external synchronization

See also

[rx_i2c_comm_send\(\)](#) Send frames to RPi5

Since

Version 1.0.0

Definition at line 966 of file [rx_i2c_comm.c](#).

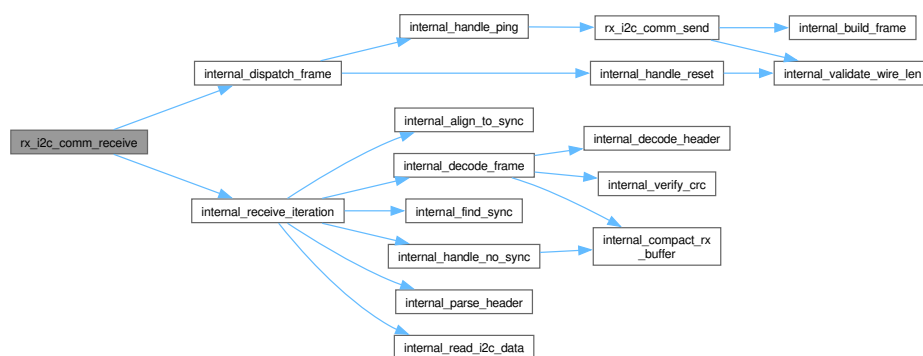
```

00967 {
00968     (void)timeout_ms; /* I2C receive is non-blocking; timeout reserved for future use */
00969
00970     if (handle == nullptr) {
00971         return k_rx_err_invalid_arg;
00972     }
00973
00974     if (frame == nullptr) {
00975         return k_rx_err_invalid_arg;
00976     }
00977
00978     if (!handle->initialized) {
00979         return k_rx_err_invalid_state;
00980     }
00981
00982     rx_err_t err = k_rx_ok;
00983
00984     for (uint32_t iterations = 0; iterations < k_i2c_comm_max_receive_iterations; iterations++) {
00985         rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
00986         if (result == k_receive_error) {
00987             return err;
00988         }
00989         if (result != k_receive_done) {
00990             continue;
00991         }
00992
00993         i2c_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
00994         if (dispatch == k_dispatch_error) {
00995             return err;
00996         }
00997         if (dispatch == k_dispatch_done) {
00998             return k_rx_ok;
00999         }
01000         /* k_dispatch_continue: loop for next frame */
01001     }
01002
01003     return k_rx_err_timeout;
01004 }

```

References [rx_i2c_comm_handle_t::initialized](#), [internal_dispatch_frame\(\)](#), [internal_receive_iteration\(\)](#), [k_dispatch_done](#), [k_dispatch_error](#), [k_i2c_comm_max_receive_iterations](#), [k_receive_done](#), and [k_receive_error](#).

Here is the call graph for this function:



4.3.3.19 rx_i2c_comm_send()

```
rx_err_t rx_i2c_comm_send (
    rx_i2c_comm_handle_t * handle,
    rx_frame_type_t type,
    uint8_t flags,
    const uint8_t * payload,
    uint32_t payload_len)  [nodiscard]
```

Send a frame over I2C (RIIC peripheral -> controller).

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation, then transmits it via riic_peripheral_write(). The RPi5 (I2C controller) must initiate a read transaction to receive data.

Parameters

in,out	handle	Initialized handle
in	type	Frame type (typically k_frame_type_response)
in	flags	Frame flags (ORed together, 0 for none)
in	payload	Pointer to payload data (may be nullptr if payload_len == 0)
in	payload_len	Payload length in bytes (0 to k_frame_max_payload)

Returns

rx_err_t Error code

Return values

k_rx_ok	Frame queued for I2C controller read
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	Not initialized
k_rx_err_invalid_size	Payload exceeds maximum frame size
k_rx_err_timeout	I2C transmit register not ready within timeout

Precondition

handle initialized via rx_i2c_comm_init()

Postcondition

On success: session TX sequence incremented

Note

I2C send requires RPi5 to initiate a read transaction

Thread safety: caller must provide external synchronization

See also

[rx_i2c_comm_receive\(\)](#) Receive frames from RPi5

Since

Version 1.0.0

Definition at line 770 of file [rx_i2c_comm.c](#).

```

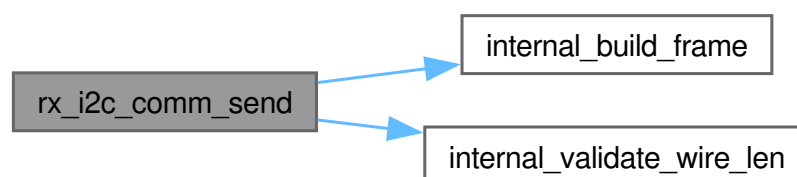
00775 {
00776     if (handle == nullptr) {
00777         return k_rx_err_invalid_arg;
00778     }
00779
00780     if (!handle->initialized) {
00781         rx_log_error(s_tag, "Handle not initialized");
00782         return k_rx_err_invalid_state;
00783     }
00784
00785     if (payload == nullptr && payload_len > 0) {
00786         return k_rx_err_invalid_arg;
00787     }
00788
00789     if (payload_len > k_frame_max_payload) {
00790         rx_log_error(s_tag, "Payload too large");
00791         return k_rx_err_invalid_size;
00792     }
00793
00794     uint16_t sequence = k_initial_sequence;
00795     rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00796     if (err != k_rx_ok) {
00797         rx_log_error(s_tag, "Session next_tx failed");
00798         return err;
00799     }
00800
00801     rx_frame_t frame = {};
00802     /* internal_build_frame only fails for invalid params; all params are validated above */
00803     (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00804
00805     uint32_t wire_len = 0;
00806     err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00807     if (err != k_rx_ok) {
00808         rx_log_error(s_tag, "Frame encode failed");
00809         return err;
00810     }
00811
00812     err = internal_validate_wire_len(wire_len);
00813     if (err != k_rx_ok) {
00814         return err;
00815     }
00816
00817     const riic_channel_t ch = {.value = handle->channel_value};
00818     err = riic_peripheral_write(ch, handle->tx_buffer, (uint16_t)wire_len);
00819     if (err != k_rx_ok) {
00820         rx_log_error(s_tag, "I2C peripheral write failed");
00821         return err;
00822     }
00823
00824     return k_rx_ok;
00825 }

```

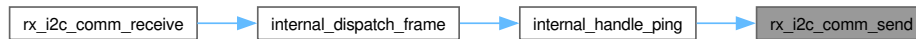
References [rx_i2c_comm_handle_t::channel_value](#), [rx_i2c_comm_handle_t::encoder](#), [rx_i2c_comm_handle_t::initial_sequence](#), [internal_build_frame\(\)](#), [internal_validate_wire_len\(\)](#), [k_initial_sequence](#), [s_tag](#), [rx_i2c_comm_handle_t::session](#), and [rx_i2c_comm_handle_t::tx_buffer](#).

Referenced by [internal_handle_ping\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4 Variable Documentation

4.3.4.1 s_tag

```
const char* const s_tag = "I2C_COMM" [static]
```

Definition at line 71 of file [rx_i2c_comm.c](#).

Referenced by [internal_decode_frame\(\)](#), [internal_dispatch_frame\(\)](#), [internal_handle_ping\(\)](#), [internal_handle_reset\(\)](#), [internal_parse_header\(\)](#), [internal_validate_wire_len\(\)](#), [rx_i2c_comm_deinit\(\)](#), [rx_i2c_comm_init\(\)](#), and [rx_i2c_comm_send\(\)](#).

4.3.4.2 s_zero_handle

```
const rx_i2c_comm_handle_t s_zero_handle = {} [static]
```

Zero-initialized I2C comm handle template for stack-safe initialization.

Static const instance used to clear [rx_i2c_comm_handle_t](#) without creating a large compound literal on the stack.

See also

[rx_i2c_comm_init\(\)](#) Uses this template to zero-initialize the comm handle

Since

Version 1.0.0

Definition at line 643 of file [rx_i2c_comm.c](#).

```
00643 {};
```

Referenced by [rx_i2c_comm_init\(\)](#).

4.4 rx_i2c_comm.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_i2c_comm.c
00003  * @brief High-Level I2C Peripheral Communication Layer Implementation
00004  *
00005  * @details
00006  * Implements reliable I2C communication by integrating the frame protocol layer
00007  * (rx_frame) with the RIIC peripheral driver (hardware.h HAL). The RX72N acts
00008  * as an I2C peripheral device, responding to address transactions from the RPi5
00009  * I2C controller. Handles frame encoding/decoding, CRC-32 validation, sequence
00010  * number management, and receive buffer handling.
00011  *
00012  * @par Implementation Architecture:
00013  * @msc
00014  * App, I2C_Comm, Frame, RIIC_HAL;
00015  * ... [label="TX Path (peripheral -> controller)"];
00016  * App => I2C_Comm [label="send(type, payload)"];
00017  * I2C_Comm => Frame [label="encode()"];
00018  * I2C_Comm => RIIC_HAL [label="riic_peripheral_write()"];
00019  * ... [label="RX Path (controller -> peripheral)"];
00020  * RIIC_HAL => I2C_Comm [label="riic_peripheral_read()"];
00021  * I2C_Comm box I2C_Comm [label="Find sync, decode"];
00022  * I2C_Comm => App [label="frame"];
00023  * @endmsc
00024  *
00025  * @par Channel Assignment:
00026  * | Channel | Constant | Purpose |
00027  * |-----|-----|-----|
00028  * | RIIC0 | k_i2c_comm_default_channel (0) | Binary protocol |
00029  *
00030  * @par Performance:
00031  * | Operation | Typical | Worst Case |
00032  * |-----|-----|-----|
00033  * | Send (64B) | 2 ms | 10 ms |
00034  * | Receive | non-block | non-block |
00035  *
00036  * @par NASA Power of 10 Compliance:
00037  * - Rule 1: [OK] No goto, setjmp, recursion
00038  * - Rule 2: [OK] Loops bounded by k_i2c_comm_max_receive_iterations
00039  * - Rule 3: [OK] No dynamic allocation
00040  * - Rule 5: [OK] Pre/post conditions on all functions
00041  * - Rule 7: [OK] All return values checked
00042  * - Rule 8: [OK] Typed enums for constants
00043  *
00044  * @par SOLID Principles:
00045  * - SRP: Handles I2C framing only, delegates I/O to RIIC HAL
00046  * - DIP: Uses hardware.h HAL abstraction
00047  *
00048  * @see rx_i2c_comm.h Public API header
00049  * @see rx_frame.h Frame encoding layer
00050  * @see hardware.h RIIC HAL
00051  *
00052  * @author Locked, Inc.
00053  * @date 2026-03-11
00054  * @copyright Copyright (c) 2026 Locked Inc.
00055  * SPDX-License-Identifier: MIT
00056  */
00057
00058 #include "rx_i2c_comm.h"
00059
00060 #include <string.h>
00061
00062 #include "rx_check.h"
00063 #include "rx_crc.h"
00064 #include "rx_log.h"
00065
00066 /**
=====
00067  * Module State
00068  *
=====
00069  */
00070
00071 static const char* const s_tag = "I2C_COMM";
00072
00073 /**
=====
00074  * Internal Helpers
00075  *
=====
00076  */
00077
00078 /**

```

```

00079 * @brief Validate that an encoded wire length fits within the I2C HAL transfer limit
00080 *
00081 * @details
00082 * Checks wire_len against k_riic_peripheral_transfer_limit and logs an error if
00083 * the limit is exceeded. Called before every riic_peripheral_write to prevent
00084 * an unchecked narrowing cast from uint32_t to uint16_t.
00085 *
00086 *
00087 *
00088 * @pre wire_len reflects the output of rx_frame_encode()
00089 * @pre k_riic_peripheral_transfer_limit is defined in rx_riic.h
00090 * @post Error logged if limit exceeded
00091 * @post Caller must propagate k_rx_err_invalid_size without calling riic_peripheral_write
00092 *
00093 * @since Version 1.0.0
00094 */
00095 RX_STATIC_TESTABLE rx_err_t internal_validate_wire_len(uint32_t wire_len)
00096 {
00097     if (wire_len > (uint32_t)k_riic_peripheral_transfer_limit) {
00098         rx_log_error(s_tag, "Encoded frame exceeds I2C HAL transfer limit; cannot send");
00099         return k_rx_err_invalid_size;
00100     }
00101     return k_rx_ok;
00102 }
00103
00104 /*
=====
00105 * Frame Header Constants
00106 *
00107 * Frame format: [SYNC(2B)][SEQ(2B)][LEN(2B)][TYPE(1B)][FLAGS(1B)]
00108 *
=====
00109 */
00110
00111 /**
00112 * @enum rx_i2c_comm_header_size_t
00113 * @brief Total encoded frame header size in bytes
00114 *
00115 * @details
00116 * Computed at compile time from the individual field size constants defined in
00117 * rx_frame.h. Used when determining how many bytes are needed to hold a complete
00118 * frame header before the payload begins.
00119 */
00120 typedef enum : uint8_t {
00121     k_frame_header_total = k_frame_sync_size + k_frame_seq_size + k_frame_len_size +
00122         k_frame_type_size + k_frame_flags_size, /**< Total header bytes */
00123 } rx_i2c_comm_header_size_t;
00124
00125 /**
00126 * @enum i2c_crc32_seed_t
00127 * @brief Initial seed value for CRC-32 output variable
00128 *
00129 * @details
00130 * Used to zero-initialize the CRC output variable passed to rx_crc32_ieee()
00131 * before the calculation begins, avoiding use of a magic literal 0U.
00132 */
00133 typedef enum : uint32_t {
00134     k_i2c_crc32_seed_initial = 0U, /**< Initial value for CRC-32 output variable */
00135 } i2c_crc32_seed_t;
00136
00137 /**
00138 * @brief Decode frame header from raw wire data
00139 *
00140 * @details
00141 * Parses the fixed-size frame header from a raw byte buffer. Validates the sync
00142 * word, extracts sequence number, payload length, type, and flags fields. Checks
00143 * that the buffer is large enough for the declared payload before returning.
00144 *
00145 *
00146 *
00147 * @pre data != nullptr
00148 * @pre frame != nullptr
00149 * @post frame->header fields populated on success
00150 * @post *offset_out contains byte offset past header if not nullptr and k_rx_ok
00151 */
00152 RX_STATIC_TESTABLE rx_err_t internal_decode_header(const uint8_t* data,
00153     const uint32_t data_len,
00154     rx_frame_t* frame,
00155     uint32_t* offset_out)
00156 {
00157     RX_ASSERT(data != nullptr, "Data pointer is nullptr");
00158     RX_ASSERT(frame != nullptr, "Frame pointer is nullptr");
00159     if (data == nullptr || frame == nullptr) {
00160         return k_rx_err_invalid_arg;
00161     }
00162
00163     if (data_len < k_frame_min_size) {

```

```

00164     return k_rx_err_invalid_size;
00165 }
00166
00167 uint32_t offset = 0;
00168 const uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00169 if (sync_word != k_frame_sync_word) {
00170     return k_rx_err_protocol_error;
00171 }
00172 offset += k_frame_sync_size;
00173
00174 frame->header.sequence = rx_frame_read_le16(&data[offset]);
00175 offset += k_frame_seq_size;
00176
00177 frame->header.length = rx_frame_read_le16(&data[offset]);
00178 offset += k_frame_len_size;
00179
00180 if (frame->header.length > k_frame_max_payload) {
00181     return k_rx_err_invalid_size;
00182 }
00183
00184 frame->header.type = data[offset];
00185 offset += k_frame_type_size;
00186
00187 frame->header.flags = data[offset];
00188 offset += k_frame_flags_size;
00189
00190 if (offset_out != nullptr) {
00191     *offset_out = offset;
00192 }
00193
00194 return k_rx_ok;
00195 }
00196
00197 /**
00198  * @brief Verify CRC-32 IEEE checksum for received frame
00199  *
00200  * @details
00201  * Reads the 4-byte CRC stored at data[offset..offset+3] (little-endian),
00202  * computes CRC-32 IEEE 802.3 over data[0..offset-1], and compares the two
00203  * values to confirm frame integrity.
00204  *
00205  *
00206  *
00207  * @pre data != nullptr
00208  * @pre offset >= (k_frame_min_size - k_frame_crc_size)
00209  * @post Frame integrity validated on k_rx_ok return
00210  * @post *crc_out contains verified CRC value if not nullptr and k_rx_ok
00211  */
00212 RX_STATIC_TESTABLE rx_err_t internal_verify_crc(const uint8_t* data,
00213         uint32_t offset,
00214         uint32_t* crc_out)
00215 {
00216     RX_ASSERT(data != nullptr, "Data pointer is nullptr");
00217     if (data == nullptr) {
00218         return k_rx_err_invalid_arg;
00219     }
00220
00221     RX_ASSERT(offset >= (k_frame_min_size - k_frame_crc_size),
00222         "CRC offset too small for valid frame");
00223     if (offset < (k_frame_min_size - k_frame_crc_size)) {
00224         return k_rx_err_invalid_arg;
00225     }
00226
00227     const uint32_t received_crc = rx_frame_read_le32(&data[offset]);
00228     uint32_t calculated_crc = k_i2c_crc32_seed_initial;
00229     /* rx_crc32_ieee only fails on null/zero-len; data and offset are validated by caller */
00230     (void)rx_crc32_ieee(data, offset, &calculated_crc);
00231
00232     if (received_crc != calculated_crc) {
00233         return k_rx_err_crc_mismatch;
00234     }
00235
00236     if (crc_out != nullptr) {
00237         *crc_out = received_crc;
00238     }
00239
00240     return k_rx_ok;
00241 }
00242
00243 /**
00244  * @enum rx_i2c_comm_sequence_constants_t
00245  * @brief Sequence number initial value constants
00246  *
00247  * @details
00248  * Provides a named constant for the sequence number starting value used when
00249  * initializing a TX sequence or building a reset-ack frame, avoiding magic 0.
00250  */

```

```

00251 typedef enum : uint16_t {
00252     k_initial_sequence = 0, /**< Initial TX/RX sequence number */
00253 } rx_i2c_comm_sequence_constants_t;
00254
00255 /**
00256  * @enum rx_i2c_comm_sync_constants_t
00257  * @brief Sync word search helper constants
00258  *
00259  * @details
00260  * Provides a named offset for the second byte of the two-byte sync word when
00261  * scanning the receive buffer for frame boundaries.
00262  */
00263 typedef enum : uint8_t {
00264     k_sync_second_byte_offset = 1, /**< Offset from first sync byte to second sync byte */
00265 } rx_i2c_comm_sync_constants_t;
00266
00267 /**
00268  * @enum i2c_frame_header_offset_t
00269  * @brief Byte offsets into the raw frame header buffer
00270  *
00271  * @details
00272  * Provides named constants for the fixed field positions within the encoded
00273  * frame header so that internal_parse_header() avoids magic number indexing.
00274  */
00275 typedef enum : uint8_t {
00276     k_hdr_len_offset = 4, /**< Payload length field byte offset within header */
00277 } i2c_frame_header_offset_t;
00278
00279 /**
=====
00280  * Internal Helpers
00281  *
=====
00282  */
00283
00284 /**
00285  * @enum rx_i2c_comm_sync_result_t
00286  * @brief Sentinel value returned by internal_find_sync() when no sync is found
00287  *
00288  * @details
00289  * Uses a signed int32_t so -1 is representable, distinguishing "not found"
00290  * from any valid non-negative buffer position.
00291  */
00292 typedef enum : int32_t {
00293     k_sync_not_found = -1, /**< Sync word not found in buffer */
00294 } rx_i2c_comm_sync_result_t;
00295
00296 /**
00297  * @enum rx_receive_result_t
00298  * @brief Return codes for internal_receive_iteration()
00299  *
00300  * @details
00301  * Allows the receive loop in rx_i2c_comm_receive() to distinguish between
00302  * "need more data", "frame ready", and "error occurred" without overloading
00303  * the rx_err_t return type.
00304  */
00305 typedef enum : uint8_t {
00306     k_receive_continue = 0, /**< Continue to next iteration */
00307     k_receive_done = 1, /**< Frame received successfully */
00308     k_receive_error = 2, /**< Error occurred, check err output */
00309 } rx_receive_result_t;
00310
00311 /**
00312  * @brief Read available data from RIIC peripheral into staging buffer
00313  *
00314  *
00315  *
00316  * @pre handle != nullptr
00317  * @pre handle->rx_buffer_len <= k_i2c_comm_rx_buffer_size
00318  * @post handle->rx_buffer_len <= k_i2c_comm_rx_buffer_size
00319  */
00320 RX_STATIC_TESTABLE rx_err_t internal_read_i2c_data(rx_i2c_comm_handle_t* handle)
00321 {
00322     if (handle == nullptr) {
00323         return k_rx_err_invalid_arg;
00324     }
00325
00326     const uint32_t space = k_i2c_comm_rx_buffer_size - handle->rx_buffer_len;
00327     if (space == 0) {
00328         return k_rx_ok;
00329     }
00330
00331     /* Clamp to the HAL transfer limit so riic_peripheral_read never receives
00332      * a length that exceeds k_riic_peripheral_transfer_limit (256 bytes). */
00333     const uint16_t read_len = (space > (uint32_t)k_riic_peripheral_transfer_limit)
00334         ? k_riic_peripheral_transfer_limit
00335         : (uint16_t)space;

```

```

00336
00337 const riic_channel_t ch      = {.value = handle->channel_value};
00338 uint16_t                bytes_read = 0;
00339 const rx_err_t          err =
00340 riic_peripheral_read(ch, handle->rx_buffer + handle->rx_buffer_len, read_len, &bytes_read);
00341 if (err != k_rx_ok) {
00342     return err;
00343 }
00344
00345 handle->rx_buffer_len += (uint32_t)bytes_read;
00346
00347 return k_rx_ok;
00348 }
00349
00350 /**
00351  * @brief Compact receive buffer by removing consumed data
00352  *
00353  *
00354  *
00355  * @pre handle != nullptr
00356  * @pre handle->rx_buffer_pos <= handle->rx_buffer_len
00357  * @post handle->rx_buffer_pos == 0
00358  */
00359 RX_STATIC_TESTABLE rx_err_t internal_compact_rx_buffer(rx_i2c_comm_handle_t* handle)
00360 {
00361     if (handle == nullptr) {
00362         return k_rx_err_invalid_arg;
00363     }
00364
00365     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00366         return k_rx_err_invalid_state;
00367     }
00368
00369     if (handle->rx_buffer_pos == 0) {
00370         return k_rx_ok;
00371     }
00372
00373     if (handle->rx_buffer_pos >= handle->rx_buffer_len) {
00374         handle->rx_buffer_len = 0;
00375         handle->rx_buffer_pos = 0;
00376     } else {
00377         const uint32_t remaining = handle->rx_buffer_len - handle->rx_buffer_pos;
00378         for (uint32_t i = 0; i < remaining; i++) {
00379             handle->rx_buffer[i] = handle->rx_buffer[handle->rx_buffer_pos + i];
00380         }
00381         handle->rx_buffer_len = remaining;
00382         handle->rx_buffer_pos = 0;
00383     }
00384
00385     return k_rx_ok;
00386 }
00387
00388 /**
00389  * @brief Search for frame sync word in receive buffer
00390  *
00391  * @details
00392  * Scans the receive buffer starting at rx_buffer_pos for the two-byte frame
00393  * sync word (k_frame_sync_word) stored in little-endian byte order. Sets
00394  * *sync_pos to the index of the matching byte or k_sync_not_found (-1).
00395  *
00396  *
00397  *
00398  * @pre handle != nullptr
00399  * @pre sync_pos != nullptr
00400  * @post *sync_pos contains the buffer index of sync word on k_rx_ok
00401  * @post *sync_pos == k_sync_not_found on k_rx_err_not_found
00402  */
00403 RX_STATIC_TESTABLE rx_err_t internal_find_sync(const rx_i2c_comm_handle_t* handle,
00404                                                int32_t* sync_pos)
00405 {
00406     if (handle == nullptr) {
00407         return k_rx_err_invalid_arg;
00408     }
00409
00410     if (sync_pos == nullptr) {
00411         return k_rx_err_invalid_arg;
00412     }
00413
00414     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00415         return k_rx_err_invalid_state;
00416     }
00417
00418     const uint8_t sync_low  = (uint8_t)(k_frame_sync_word & k_rx_byte_mask);
00419     const uint8_t sync_high = (uint8_t)(k_frame_sync_word » k_rx_le16_high_shift);
00420
00421     for (uint32_t i = handle->rx_buffer_pos; i + k_sync_second_byte_offset < handle->rx_buffer_len;
00422         i++) {

```

```

00423     if (handle->rx_buffer[i] == sync_low &&
00424         handle->rx_buffer[i + k_sync_second_byte_offset] == sync_high) {
00425         *sync_pos = (int32_t)i;
00426         return k_rx_ok;
00427     }
00428 }
00429
00430 *sync_pos = k_sync_not_found;
00431 return k_rx_err_not_found;
00432 }
00433
00434 /**
00435  * @brief Handle case when no sync word is found in buffer
00436  *
00437  * @details
00438  * When no sync word is present, advances the buffer position by one byte if
00439  * the buffer is full (to make room for new data), then compacts the buffer by
00440  * discarding consumed bytes from the front.
00441  *
00442  *
00443  *
00444  * @pre handle != nullptr
00445  * @pre handle->rx_buffer_pos <= handle->rx_buffer_len
00446  * @post Buffer compacted; rx_buffer_pos == 0 on success
00447  * @post No sync word present in remaining buffer data
00448  */
00449 RX_STATIC_TESTABLE rx_err_t internal_handle_no_sync(rx_i2c_comm_handle_t* handle)
00450 {
00451     if (handle == nullptr) {
00452         return k_rx_err_invalid_arg;
00453     }
00454
00455     if (handle->rx_buffer_len >= k_i2c_comm_rx_buffer_size) {
00456         handle->rx_buffer_pos++;
00457     }
00458
00459     (void)internal_compact_rx_buffer(handle);
00460
00461     return k_rx_ok;
00462 }
00463
00464 /**
00465  * @brief Decode a complete frame from buffer
00466  *
00467  * @details
00468  * Reads total_size bytes from the current buffer position, decodes the header,
00469  * copies the payload, and verifies the CRC-32. Advances rx_buffer_pos by
00470  * total_size regardless of success or failure, then compacts the buffer.
00471  *
00472  *
00473  *
00474  * @pre handle != nullptr and initialized
00475  * @pre frame != nullptr
00476  * @pre total_size bytes available starting at handle->rx_buffer_pos
00477  * @post rx_buffer_pos advanced by total_size; buffer compacted
00478  * @post *frame contains valid decoded frame on k_rx_ok
00479  */
00480 RX_STATIC_TESTABLE rx_err_t internal_decode_frame(rx_i2c_comm_handle_t* handle,
00481                                                    rx_frame_t* frame,
00482                                                    const uint32_t total_size)
00483 {
00484     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00485     uint32_t offset = 0;
00486
00487     /* internal_parse_header validates the header before this call;
00488      * internal_decode_header cannot fail for pre-validated data. */
00489     (void)internal_decode_header(hdr, total_size, frame, &offset);
00490
00491     if (frame->header.length > 0) {
00492         for (uint16_t i = 0; i < frame->header.length; i++) {
00493             frame->payload[i] = hdr[offset + i];
00494         }
00495         offset += frame->header.length;
00496     }
00497
00498     rx_err_t err = internal_verify_crc(hdr, offset, &frame->crc);
00499     if (err != k_rx_ok) {
00500         rx_log_error(s_tag, "Frame CRC check failed");
00501     }
00502
00503     handle->rx_buffer_pos += total_size;
00504     (void)internal_compact_rx_buffer(handle);
00505
00506     return err;
00507 }
00508
00509 /**

```

```

00510 * @brief Align buffer position to sync word if found
00511 *
00512 * @details
00513 * Advances rx_buffer_pos to sync_pos when sync_pos is ahead of the current
00514 * position, discarding any bytes before the sync word.
00515 *
00516 *
00517 * @pre handle != nullptr
00518 * @pre sync_pos >= 0
00519 * @post handle->rx_buffer_pos >= (uint32_t)sync_pos
00520 * @post No bytes before sync_pos remain accessible
00521 */
00522 RX_STATIC_TESTABLE void internal_align_to_sync(rx_i2c_comm_handle_t* handle, const int32_t sync_pos)
00523 {
00524     if ((uint32_t)sync_pos > handle->rx_buffer_pos) {
00525         handle->rx_buffer_pos = (uint32_t)sync_pos;
00526     }
00527 }
00528
00529 /**
00530 * @brief Parse frame header and validate payload length
00531 *
00532 * @details
00533 * Reads the payload length field from the frame header at the current buffer
00534 * position and validates it against k_frame_max_payload. Computes the total
00535 * expected frame size if the length is valid, or advances the buffer position
00536 * past the sync word if invalid.
00537 *
00538 *
00539 *
00540 * @pre handle != nullptr with at least k_frame_header_total bytes available
00541 * @pre payload_len != nullptr
00542 * @pre total_size != nullptr
00543 * @post On k_rx_ok: *total_size == k_frame_header_total + *payload_len + k_frame_crc_size
00544 * @post On k_rx_err_invalid_size: rx_buffer_pos advanced by k_frame_sync_size
00545 */
00546 RX_STATIC_TESTABLE rx_err_t internal_parse_header(rx_i2c_comm_handle_t* handle,
00547                                                    uint16_t* payload_len,
00548                                                    uint32_t* total_size)
00549 {
00550     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00551     *payload_len = rx_frame_read_le16(&hdr[k_hdr_len_offset]);
00552
00553     if (*payload_len > k_frame_max_payload) {
00554         rx_log_warn(s_tag, "Invalid payload length, skipping");
00555         handle->rx_buffer_pos += k_frame_sync_size;
00556         return k_rx_err_invalid_size;
00557     }
00558
00559     *total_size = k_frame_header_total + *payload_len + k_frame_crc_size;
00560     return k_rx_ok;
00561 }
00562
00563 /**
00564 * @brief Process one iteration of the frame receive loop
00565 *
00566 * @details
00567 * Performs a single pass of the receive state machine: reads available I2C data,
00568 * searches for a sync word, validates the header, and decodes a complete frame
00569 * when enough bytes are present.
00570 *
00571 *
00572 *
00573 * @pre handle != nullptr and handle->initialized == true
00574 * @pre frame != nullptr
00575 * @pre err != nullptr
00576 * @post On k_receive_done: *frame contains a valid, CRC-verified frame
00577 * @post On k_receive_error: *err contains the failure code
00578 */
00579 RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration(rx_i2c_comm_handle_t* handle,
00580                                                                    rx_frame_t* frame,
00581                                                                    rx_err_t* err)
00582 {
00583     *err = internal_read_i2c_data(handle);
00584     /* k_rx_err_timeout means the HAL had no new bytes this iteration; treat as
00585      * "buffer unchanged" and fall through to the sync-search so the caller can
00586      * re-attempt up to k_i2c_comm_max_receive_iterations times before giving up. */
00587     if (*err != k_rx_ok && *err != k_rx_err_timeout) {
00588         return k_receive_error;
00589     }
00590
00591     if (handle->rx_buffer_len == handle->rx_buffer_pos) {
00592         *err = k_rx_err_no_data;
00593         return k_receive_error;
00594     }
00595
00596     int32_t sync_pos = k_sync_not_found;

```

```

00597 *err          = internal_find_sync(handle, &sync_pos);
00598 if (*err == k_rx_err_not_found) {
00599     (void)internal_handle_no_sync(handle);
00600     *err = k_rx_err_no_data;
00601     return k_receive_error;
00602 }
00603
00604 internal_align_to_sync(handle, sync_pos);
00605
00606 const uint32_t available = handle->rx_buffer_len - handle->rx_buffer_pos;
00607 if (available < k_frame_header_total) {
00608     *err = k_rx_err_no_data;
00609     return k_receive_error;
00610 }
00611
00612 uint32_t total_size = 0;
00613 uint16_t payload_len = 0;
00614 *err          = internal_parse_header(handle, &payload_len, &total_size);
00615 if (*err == k_rx_err_invalid_size) {
00616     /* Invalid payload length; buffer advanced, continue to find next sync */
00617     *err = k_rx_ok;
00618     return k_receive_continue;
00619 }
00620
00621 if (available < total_size) {
00622     *err = k_rx_err_no_data;
00623     return k_receive_error;
00624 }
00625
00626 *err = internal_decode_frame(handle, frame, total_size);
00627 return (*err != k_rx_ok) ? k_receive_error : k_receive_done;
00628 }
00629
00630 /*
=====
00631 * Initialization
00632 *
=====
00633 */
00634 /**
00635  * @var s_zero_handle
00636  * @brief Zero-initialized I2C comm handle template for stack-safe initialization
00637  * @details Static const instance used to clear rx_i2c_comm_handle_t without creating
00638  *          a large compound literal on the stack.
00639  * @see rx_i2c_comm_init() Uses this template to zero-initialize the comm handle
00640  * @since Version 1.0.0
00641  */
00642 /*
00643 static const rx_i2c_comm_handle_t s_zero_handle = {};
00644
00645 rx_err_t rx_i2c_comm_init(rx_i2c_comm_handle_t* handle, const rx_i2c_comm_config_t* config)
00646 {
00647     if (handle == nullptr) {
00648         return k_rx_err_invalid_arg;
00649     }
00650
00651     *handle = s_zero_handle;
00652
00653     if (config == nullptr || config->session == nullptr) {
00654         return k_rx_err_invalid_arg;
00655     }
00656     handle->session      = config->session;
00657     handle->channel_value = config->channel.value;
00658     handle->device_addr  = config->device_addr.value;
00659
00660     /* encoder/decoder init only fail for nullptr, which is impossible since handle is non-null */
00661     (void)rx_frame_encoder_init(&handle->encoder);
00662     (void)rx_frame_decoder_init(&handle->decoder);
00663
00664     /* Initialize RIIC in peripheral mode */
00665     rx_err_t err = riic_init_peripheral(config->channel, config->device_addr);
00666     if (err != k_rx_ok) {
00667         rx_log_error(s_tag, "Failed to init RIIC peripheral mode");
00668
00669         (void)rx_frame_encoder_deinit(&handle->encoder);
00670         (void)rx_frame_decoder_deinit(&handle->decoder);
00671         return err;
00672     }
00673
00674     handle->rx_buffer_len = 0;
00675     handle->rx_buffer_pos = 0;
00676     handle->initialized    = 1U;
00677     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START (assigned 1U on line above) */
00678     RX_ASSERT_POST(handle->initialized, "Handle initialization failed");
00679     /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00680
00681     rx_log_debug(s_tag, "I2C comm initialized");

```

```

00682     return k_rx_ok;
00683 }
00684
00685 rx_err_t rx_i2c_comm_deinit(rx_i2c_comm_handle_t* handle)
00686 {
00687     if (handle == nullptr) {
00688         return k_rx_err_invalid_arg;
00689     }
00690
00691     RX_ASSERT_PRE(handle->initialized, "Attempt to deinitialize uninitialized I2C comm handle");
00692
00693     (void)rx_frame_encoder_deinit(&handle->encoder);
00694     (void)rx_frame_decoder_deinit(&handle->decoder);
00695
00696     /* Hardware (RIIC peripheral) cleanup is the caller's responsibility.
00697      * The hardware.h HAL does not expose a riic_deinit() function; the RIIC
00698      * module stop bit (MSTPCRB) must be set by the caller if power-gating is
00699      * needed after deinitializing this comm layer. */
00700
00701     handle->initialized = 0U;
00702
00703     rx_log_debug(s_tag, "I2C comm deinitialized");
00704     return k_rx_ok;
00705 }
00706
00707 /*
=====
00708 * Send API
00709 *
=====
00710 */
00711 /**
00712  * @brief Build frame structure from individual parameters
00713  *
00714  * @details
00715  * Populates all header fields of *frame and copies the payload bytes into
00716  * frame->payload. Used by rx_i2c_comm_send() to construct outgoing frames
00717  * before encoding and transmission.
00718  *
00719  *
00720  *
00721  *
00722  * @pre frame != nullptr
00723  * @pre payload != nullptr || payload_len == 0
00724  * @post frame->header.sequence, type, length, flags are set
00725  * @post frame->payload[0..payload_len-1] copied from payload on k_rx_ok
00726  */
00727 RX_STATIC_TESTABLE rx_err_t internal_build_frame(rx_frame_t* frame,
00728           const uint16_t sequence,
00729           const rx_frame_type_t type,
00730           const uint8_t flags,
00731           const uint8_t* payload,
00732           const uint32_t payload_len)
00733 {
00734     RX_ASSERT(frame != nullptr, "Frame pointer is nullptr");
00735     if (frame == nullptr) {
00736         return k_rx_err_invalid_arg;
00737     }
00738
00739     RX_ASSERT(payload != nullptr || payload_len == 0, "Payload nullptr but payload_len > 0");
00740     if (payload == nullptr && payload_len > 0) {
00741         return k_rx_err_invalid_arg;
00742     }
00743
00744     if (payload_len > k_frame_max_payload) {
00745         return k_rx_err_invalid_size;
00746     }
00747
00748     frame->header.sequence = sequence;
00749     frame->header.length   = (uint16_t)payload_len;
00750     frame->header.type     = (uint8_t)type;
00751     frame->header.flags    = flags;
00752
00753     /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START
00754      * gcov -fprofile-arcs quirk with short-circuit && across multiple TUs --
00755      * test_i2c_internal_build_frame_with_payload calls this function with
00756      * payload != nullptr and payload_len = 4, asserts the 4-byte memcpy
00757      * happened, and passes. The `payload_len > 0 TRUE' sub-branch of the &&
00758      * still reports 0 hits in the merged .info. The behaviour is exercised;
00759      * only the counting is wrong. */
00760     if (payload != nullptr && payload_len > 0) {
00761         for (uint32_t i = 0; i < payload_len; i++) {
00762             frame->payload[i] = payload[i];
00763         }
00764     }
00765     /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00766

```

```

00767     return k_rx_ok;
00768 }
00769
00770 rx_err_t rx_i2c_comm_send(rx_i2c_comm_handle_t* handle,
00771                          rx_frame_type_t type,
00772                          uint8_t flags,
00773                          const uint8_t* payload,
00774                          uint32_t payload_len)
00775 {
00776     if (handle == nullptr) {
00777         return k_rx_err_invalid_arg;
00778     }
00779     if (!handle->initialized) {
00780         rx_log_error(s_tag, "Handle not initialized");
00781         return k_rx_err_invalid_state;
00782     }
00783 }
00784
00785 if (payload == nullptr && payload_len > 0) {
00786     return k_rx_err_invalid_arg;
00787 }
00788
00789 if (payload_len > k_frame_max_payload) {
00790     rx_log_error(s_tag, "Payload too large");
00791     return k_rx_err_invalid_size;
00792 }
00793
00794 uint16_t sequence = k_initial_sequence;
00795 rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00796 if (err != k_rx_ok) {
00797     rx_log_error(s_tag, "Session next_tx failed");
00798     return err;
00799 }
00800
00801 rx_frame_t frame = {};
00802 /* internal_build_frame only fails for invalid params; all params are validated above */
00803 (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00804
00805 uint32_t wire_len = 0;
00806 err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00807 if (err != k_rx_ok) {
00808     rx_log_error(s_tag, "Frame encode failed");
00809     return err;
00810 }
00811
00812 err = internal_validate_wire_len(wire_len);
00813 if (err != k_rx_ok) {
00814     return err;
00815 }
00816
00817 const riic_channel_t ch = {.value = handle->channel_value};
00818 err = riic_peripheral_write(ch, handle->tx_buffer, (uint16_t)wire_len);
00819 if (err != k_rx_ok) {
00820     rx_log_error(s_tag, "I2C peripheral write failed");
00821     return err;
00822 }
00823
00824 return k_rx_ok;
00825 }
00826
00827 /*
=====
00828 * Receive API
00829 *
=====
00830 */
00831
00832 /**
00833  * @brief Handle a received PING frame by sending a PONG response
00834  *
00835  * @details
00836  * Sends a PONG frame echoing the received PING payload back to the I2C
00837  * controller. Called automatically by rx_i2c_comm_receive() when a PING
00838  * frame is decoded.
00839  *
00840  *
00841  *
00842  * @pre handle != nullptr and handle->initialized == true
00843  * @pre frame != nullptr and frame->header.type == k_frame_type_ping
00844  * @post PONG frame transmitted over I2C peripheral write
00845  * @post Session TX sequence number incremented
00846  */
00847 RX_STATIC_TESTABLE rx_err_t internal_handle_ping(rx_i2c_comm_handle_t* handle,
00848                                                  const rx_frame_t* frame)
00849 {
00850     rx_err_t pong_err = rx_i2c_comm_send(handle,
00851                                         k_frame_type_pong,

```

```

00852             k_frame_flag_none,
00853             frame->payload,
00854             frame->header.length);
00855 if (pong_err != k_rx_ok) {
00856     rx_log_error(s_tag, "Failed to send PONG response");
00857     return pong_err;
00858 }
00859 rx_log_debug(s_tag, "Auto-responded with PONG");
00860 return k_rx_ok;
00861 }
00862
00863 /**
00864  * @brief Handle a received RESET frame by sending RESET_ACK and resetting session
00865  *
00866  * @details
00867  * Constructs and transmits a RESET_ACK frame over the I2C peripheral write
00868  * channel, then resets the session state machine. Called automatically by
00869  * rx_i2c_comm_receive() when a RESET frame is decoded.
00870  *
00871  *
00872  *
00873  * @pre handle != nullptr and handle->initialized == true
00874  * @pre handle->session != nullptr and initialized
00875  * @post RESET_ACK frame transmitted over I2C peripheral write
00876  * @post handle->session reset to initial state on k_rx_ok
00877  */
00878 RX_STATIC_TESTABLE rx_err_t internal_handle_reset(rx_i2c_comm_handle_t* handle)
00879 {
00880     uint16_t reset_ack_seq = k_initial_sequence;
00881     const rx_err_t seq_err = rx_session_next_tx(handle->session, &reset_ack_seq);
00882     if (seq_err != k_rx_ok) {
00883         return seq_err;
00884     }
00885
00886     rx_frame_t reset_ack_frame = {};
00887     /* rx_frame_create_reset_ack only fails for nullptr; reset_ack_frame is on the stack */
00888     (void)rx_frame_create_reset_ack(&reset_ack_frame, reset_ack_seq);
00889
00890     uint32_t wire_len = 0;
00891     /* rx_frame_encode only fails for nullptr/uninitialized args; all are valid here */
00892     (void)rx_frame_encode(&handle->encoder, &reset_ack_frame, handle->tx_buffer, &wire_len);
00893
00894     /* Reset ACK frame is always within I2C transfer limits (fixed small size) */
00895     (void)internal_validate_wire_len(wire_len);
00896
00897     const riic_channel_t ch = {.value = handle->channel_value};
00898     rx_err_t write_err = riic_peripheral_write(ch, handle->tx_buffer, (uint16_t)wire_len);
00899     if (write_err != k_rx_ok) {
00900         rx_log_error(s_tag, "Failed to send RESET_ACK");
00901         return write_err;
00902     }
00903     rx_log_debug(s_tag, "Auto-responded with RESET_ACK");
00904
00905     /* rx_session_reset only fails for nullptr; handle->session is validated by caller */
00906     (void)rx_session_reset(handle->session);
00907
00908     return k_rx_ok;
00909 }
00910
00911 /**
00912  * @enum i2c_dispatch_result_t
00913  * @brief Return codes for internal_dispatch_frame()
00914  *
00915  * @details
00916  * Distinguishes between "frame consumed internally, continue loop",
00917  * "frame dispatched to caller as data", and "error during dispatch".
00918  */
00919 typedef enum : uint8_t {
00920     k_dispatch_continue = 0, /**< Control frame consumed; continue receive loop */
00921     k_dispatch_done = 1, /**< Data frame validated; return to caller */
00922     k_dispatch_error = 2, /**< Error during dispatch; check err output */
00923 } i2c_dispatch_result_t;
00924
00925 /**
00926  * @brief Dispatch a decoded frame by type (PING/RESET/PONG/data)
00927  *
00928  *
00929  *
00930  * @pre handle != nullptr and initialized
00931  * @pre frame != nullptr with valid decoded header
00932  * @pre err != nullptr
00933  * @post On k_dispatch_continue: PONG/RESET_ACK may have been sent
00934  * @post On k_dispatch_done: session RX sequence updated
00935  */
00936 static i2c_dispatch_result_t
00937 internal_dispatch_frame(rx_i2c_comm_handle_t* handle, rx_frame_t* frame, rx_err_t* err)
00938 {

```

```

00939 /* PING: auto-send PONG, continue loop for next frame */
00940 if (frame->header.type == k_frame_type_ping) {
00941     *err = internal_handle_ping(handle, frame);
00942     return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00943 }
00944
00945 /* RESET: send RESET_ACK, reset session, continue loop */
00946 if (frame->header.type == k_frame_type_reset) {
00947     *err = internal_handle_reset(handle);
00948     return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00949 }
00950
00951 /* PONG / RESET_ACK: consume silently */
00952 if (frame->header.type == k_frame_type_pong || frame->header.type == k_frame_type_reset_ack) {
00953     return k_dispatch_continue;
00954 }
00955
00956 /* Data frame: validate sequence via session */
00957 rx_session_validate_result_t validate_result = k_session_validate_fail;
00958 *err = rx_session_validate_rx(handle->session, frame->header.sequence, &validate_result);
00959 if (*err != k_rx_ok) {
00960     rx_log_error(s_tag, "Session validate_rx returned error");
00961     return k_dispatch_error;
00962 }
00963 return k_dispatch_done;
00964 }
00965
00966 rx_err_t rx_i2c_comm_receive(rx_i2c_comm_handle_t* handle, rx_frame_t* frame, uint32_t timeout_ms)
00967 {
00968     (void)timeout_ms; /* I2C receive is non-blocking; timeout reserved for future use */
00969
00970     if (handle == nullptr) {
00971         return k_rx_err_invalid_arg;
00972     }
00973
00974     if (frame == nullptr) {
00975         return k_rx_err_invalid_arg;
00976     }
00977
00978     if (!handle->initialized) {
00979         return k_rx_err_invalid_state;
00980     }
00981
00982     rx_err_t err = k_rx_ok;
00983
00984     for (uint32_t iterations = 0; iterations < k_i2c_comm_max_receive_iterations; iterations++) {
00985         rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
00986         if (result == k_receive_error) {
00987             return err;
00988         }
00989         if (result != k_receive_done) {
00990             continue;
00991         }
00992
00993         i2c_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
00994         if (dispatch == k_dispatch_error) {
00995             return err;
00996         }
00997         if (dispatch == k_dispatch_done) {
00998             return k_rx_ok;
00999         }
01000     } /* k_dispatch_continue: loop for next frame */
01001 }
01002
01003 return k_rx_err_timeout;
01004 }

```